COLLEGE DE PARIS SUPÉRIEUR

Syllabus de formation

BACHELOR 3 - Algorithmique
Décisionnels : listes, tris et arbres

Semestre 1

B3COM01 : DEVELOPPEMENT & TEHCNOLOGIE III

B3COM0101 : Algorithmique Décisionnels : listes, tris et arbres	
Intitulé de la Matière	**Algorithmique Décisionnels : listes, tris et arbres**
Enseignant de la matière	M. AJAVON A. ELOM
Code	B3COM0101
Volume Horaire	28h
Crédits	3
Niveau	B3
Semestre	1
Prérequis	Notions de base en Algorithme
Description du contenu de la matière	Le but de cette matière est d'approfondir la compréhension des structures de données (listes, arbres) et des algorithmes de tri dans le contexte des décisions algorithmiques.
Objectif général	Ce cours a pour but d'approfondir la compréhension des structures de données (listes, arbres) et des algorithmes de tri dans le contexte des décisions algorithmiques. Les étudiants apprendront comment ces structures et algorithmes sont utilisés pourrésoudre des problèmes de décision et d'optimisation.Le cours aborde également les concepts de complexité algorithmique et d'optimisation des ressources, essentiels dans la conception d'algorithmes efficaces.
Objectifs spécifiques (Savoirs et savoir-faire envisagés)	Au terme de ce cours, il/elle doit être capable de : • Comprendre les structures de données de listes et arbres et leurs applications. • Implémenter et analyser des algorithmes de tri et de recherche. • Utiliser des structures de données et des algorithmes pour résoudre des problèmes décisionnels. • Optimiser les performances des algorithmes en tenant compte des contraintes de temps et d'espace. • Appliquer ces concepts dans des problèmes réels nécessitant une prise de décision.

Contenu du cours	• Les bases de l'analyse algorithmique • Stratégies de résolution de problèmes • Concept de complexité algorithmique • Analyse d'algorithmes de tri • Analyse d'algorithmes de listes • Algorithmique des arbres
Approches pédagogiques	• La méthode démonstrative ; • La méthode interrogative ou maïeutique ; • La méthode active ; • La méthode expérientielle ; • TP dirigés et corrigés
Méthode d'évaluation	• Contrôle(s) continu(s) (TD ou TP) - 50% de la note finale • Examen final ou Projet Final – 50% de la note finale

Chapitre 01 : Les bases de l'analyse algorithmique

I. Introduction

I.1. Notion d'algorithme

Dans la vie courante, un algorithme peut prendre la forme :

- d'une recette de cuisine;
- d'un itinéraire routier;
- d'un mode d'emploi, etc.

Une recette de cuisine, par exemple, est un algorithme : à partir des ingrédients, elle explique comment parvenir au plat. De même, un itinéraire routier explique comment, à partir d'une position initiale, rejoindre une position finale en un certain nombre d'étapes … .

Exemple : Préparer la pâte à tarte

 Les ingrédients :

250 g de farine ;

50 g de beurre ;

1 verre de lait.

 Les actions élémentaires à réaliser

Début

Incorporer le beurre dans la farine
Pétrir le mélange jusqu'à ce qu'il soit homogène
Ajouter du lait
Mélanger
Si la pâte est trop sèche, alors ajouter du lait
Si la pâte à une bonne consistance, alors la laisser reposer une demi heure
Faire cuire la pâte

Fin

Un algorithme sert à transmettre un savoir faire. Il décrit les étapes à suivre pour réaliser un travail. Tout comme le savoir-faire du cuisinier se transmet sous la forme d'une recette, celui d'un informaticien se transmet sous la forme d'un algorithme.

I.2. **Définition d'un algorithme**

Le mot « algorithme » provient de la forme latine (Algorismus) du nom du mathématicien arabe AL KHWARIZMI. Ce dernier formula une première définition: « Un algorithme est une séquence d'opérations visant à la résolution d'un problème en un temps fini.» Nous pouvons adopter la définition suivante : un algorithme est la description de la méthode de résolution d'un problème quelconque en utilisant des instructions élémentaires. Ces instructions deviennent compréhensibles par l'ordinateur lors de la traduction de l'algorithme en un programme.

II. **Structure générale d'un algorithme**

Un algorithme est composé de trois parties principales (figure 2) :

- l'en-tête : cette partie sert à donner un nom à l'algorithme. Elle est précédée par le mot Algorithme ;
- la partie déclarative : dans cette partie, on déclare les différents objets que l'algorithme utilise (constantes, variables, etc.) ;
- le corps de l'algorithme : cette partie contient les instructions de l'algorithme. Elle est délimitée par les mots Début et Fin.

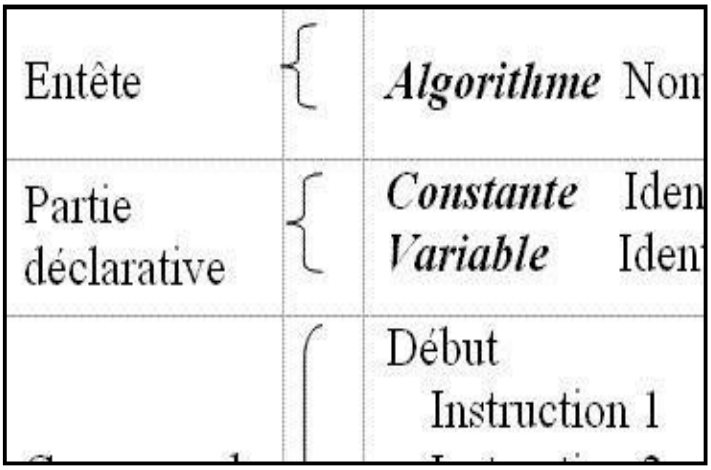


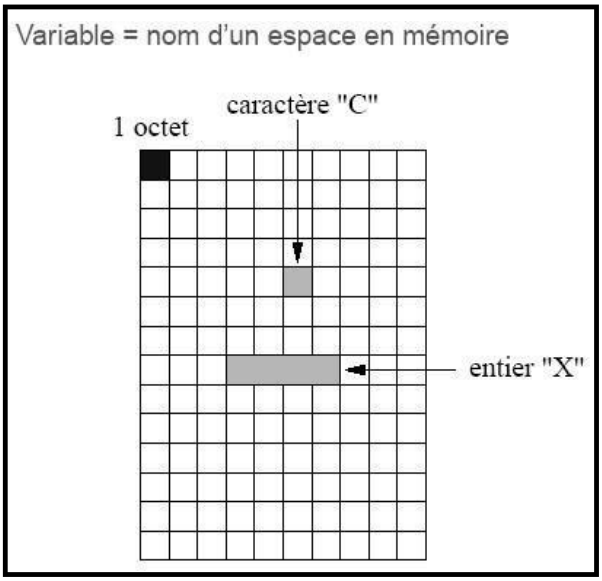
Figure 2 : Structure d'un algorithme

III. Les variables et les constantes

III.1. Notion de variable

Les données ainsi que les résultats des calculs intermédiaires ou finaux, sont rangés dans des cases mémoires qui correspondent à des variables.

Ainsi, une variable (figure suivante) est rangée dans un emplacement mémoire nommé, de taille fixe (ou non) prenant au cours du déroulement de l'algorithme, un nombre indéfini de valeurs différentes



III.2. Déclaration des variables

La partie déclaration consiste à énumérer toutes les variables dont on aura besoin au cours de l'algorithme.

Chaque déclaration doit comporter le nom de la variable (identificateur) et son type.

Syntaxe :

Variable identificateur : type Exemples :

Variable surface : réel
Variable a : entier
Variable a, b, c, d : entiers
Variable Nom_Prenom : chaîne Variable

- Identificateur

Un identificateur est le nom donné à une variable, une fonction, etc. Ce nom doit obligatoirement commencer par une lettre suivie d'une suite de lettres et de chiffres et il ne doit pas contenir d'espace.

- Types de données

Le type d'une variable est l'ensemble des valeurs qu'elle peut prendre. Par exemple, une variable de type logique (booléen) peut prendre les valeurs Vrai ou Faux.

Les différents types utilisés en algorithmique :

- Type Entier sert à manipuler les nombres entiers positifs ou négatifs. Par exemple : 5, -15, etc.
- Type Réel quant à lui sert à manipuler les nombres à virgule. Par exemple : 3.14, -15.5, etc.
- Type Caractère permet de manipuler des caractères alphabétiques et numériques. Par exemple : 'a', 'A', 'z', ' ?', '1', '2', etc.
- Type Chaîne sert à manipuler des chaînes de caractères permettant de représenter des mots ou des phrases. Par exemple : "bonjour", "Monsieur", etc.
- Type Logique (Booléen) : utilise les expressions logiques. Il n'y a que deux valeurs booléennes : Vrai et Faux.

Exemple :

Variables n : entier r : réel a, b : logiques
nom_etudiant : chaîne

A un type donné, correspond un ensemble d'opérations définies pour ce type:

Type	Opérations possibles	Symbole ou mot clé correspondant
Entier	Addition	+
	Soustraction	-
	Multiplication	*
	Division	/
	Division entière	(DIV en VB et % en C)
	Modulo (le reste de la division entière)	(MOD en VB)
	x exposant y	^ : en vb et pow(x,y) en C
	Comparaisons	<, =, >, <=, >=, ≠
En algorithmique nous symbolisons la division entière par DIV et le reste de la division entière par MOD		
Réel	Addition	+
	Soustraction	-
	Multiplication	*
	Division	/
	Exposant	^
	Comparaisons	<, =, >, <=, >=, ≠

Caractère	Comparaisons	<, =, >, <=, >=, ≠
Chaîne	Concaténation Comparaison	(+ , & : en VB) <, =, >, <=, >=, ≠
Booléen	Logiques	ET, OU, NON et OU_{ex}

Exemple :

5 / 2 = 2.5
5 Div 2 = 2
5 Mod 2 = 1
5 ^ 2 = 25

"Bonjour" & " " & "Monsieur" donne "Bonjour Monsieur" L'expression 5 > 2 est Vraie.

L'expression 7 < 4 est fausse.

Les opérations définies pour le type booléen sont :

- Le ET logique (and)
- Le OU logique (Or)
- Le NON logique (not)
- Le OU_{ex} (Ou exclusif appelée en VB Xor)

Nous résumons dans une table de vérité les résultats obtenus suivant les valeurs de deux opérandes :

P	Q	Non P	Non Q	P et Q	P ou Q	P ou_{ex} Q
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	1	0

Lois de De Morgan

Non (P et Q) □ (non P) ou (non Q)
Non (P ou Q) □ (non P) et (non Q)

III.3. **Les constantes**

Comme une variable, à une constante correspond un emplacement mémoire réservé auquel on accède par le nom qui lui a été attribué, mais dont la valeur stockée ne sera jamais modifiée au cours du programme.

Syntaxe :

Constante NOM_DE_LA_CONSTANTE = valeur Exemple :

Constante PI = 3.14

IV. Les instructions de base

Une instruction est une action élémentaire commandant à la machine un calcul, ou une communication avec l'un de ses périphériques d'entrées ou de sorties. Les instructions de base sont :

IV.1. L'instruction d'affectation

L'affectation permet d'affecter une valeur à une variable. Elle est symbolisée en algorithmique par "←".
Le signe "←" précise le sens de l'affectation.

Syntaxe :

Variable ← Expression

Expression peut être soit :

- identificateur ;
- constante ;
- expression arithmétique ;
- expression logique.

Sémantique :

Une affectation peut être définie en deux étapes :

- évaluation de l'expression qui se trouve dans la partie droite de l'affectation ;
- placement de cette valeur dans la variable.

Exemple :

```
Algorithme Calcul
Variables  A, B, C, D : entier
Début
A  ← 10
B  ← 30
C  ←  A+B
D  ← C*A

Fin
```

Note : Les lignes sont numérotées pour faciliter l'explication.

Nous pouvons expliquer ce qui ce passe par le tableau suivant :

Variable	N° de ligne					
	1	2	3	4	5	6
A	?	?	10	10	10	10
B	?	?	?	30	30	30
C	?	?	?	?	40	40
D	?	?	?	?	?	400

Remarque :

Les variables numériques ne sont pas forcément initialisées à zéro.
Leur valeur peut être n'importe quoi. C'est la raison de la présence du point d'interrogation avant qu'une première valeur ne soit affectée à la variable.

Exemple :

```
Algorithme Logique
Variables  A, B, C : Booléen
Début
A← Vrai
B←Faux
C←A et B
Fin
```

Variable	N° de ligne				
	1	2	3	4	5
A	?	?	Vrai	Vrai	Vrai
B	?	?	?	Faux	Faux
C	?	?	?	?	Faux

IV.2. L'instruction d'entrée

L'instruction d'entrée ou de lecture donne la main à l'utilisateur pour saisir une donnée au clavier. La valeur saisie sera affectée à une variable.

Syntaxe : Lire (identificateur)

Exemples :

Lire(A)

Lire(A, B, C)

L'instruction Lire(A) permet à l'utilisateur de saisir une valeur au clavier. Cette valeur sera affectée à la variable A.

Remarque :

Lorsque le programme rencontre cette instruction, l'exécution

s'interrompt et attend que l'utilisateur tape une valeur. Cette valeur est rangée en mémoire dans la variable désignée.

IV.3. L'instruction de sortie

Avant de lire une variable, il est conseillé d'écrire des libellés à l'écran, afin de prévenir l'utilisateur de ce qu'il doit frapper (sinon, l'utilisateur passe son temps à se demander ce que l'ordinateur attend de lui). L'instruction de sortie (d'écriture) permet d'afficher des informations à l'écran.

Syntaxe :

Ecrire (expression)

Expression peut être une valeur, un résultat, un message, le contenu d'une variable, etc.

Exemple 1 :

Ecrire (A)

Cette instruction permet d'afficher à l'écran la valeur de la variable A.

Exemple 2 :

A □ 2

Ecrire ("La valeur de A est = ", A) La dernière instruction affiche à l'écran : La valeur de A est = 2

Exercice : Calcul PTTC

Ecrire un algorithme qui permet de saisir le prix HT (PHT) d'un article et de calculer son prix total TTC (PTTC). TVA = 20%.

Solution :

```
Algorithme      Calcul_PTTC
Variables  PHT, PTTC : réel
Constante TVA = 0.2
Début
Ecrire ("Entrez le prix hors taxes : ")
Lire (PHT)
PTTC □ PHT + (PHT * TVA)
   Ecrire ("Le prix TTC est ", PTTC) Fin
```

Explication de l'algorithme :

N° de ligne	Explication
0	Déclare un algorithme dont le nom est Calcul_PTTC
1	Déclare les différentes variables utilisées par l'algorithme
2	Déclare la constante TVA
3	Marque le début des traitements effectués par l'algorithme.
4	Affiche à l'écran le message : Entrez le prix hors taxes :
5	Permet à l'utilisateur de saisir une valeur au clavier qui sera affectée à la variable PHT.
6	Calcul le prix TTC et affecte le résultat à la variable PTTC
7	Affiche le résultat à l'écran
8	Marque la fin de l'algorithme

V. **Les commentaires**

Lorsqu'un algorithme devient long, il est conseillé d'ajouter des lignes de commentaires dans l'algorithme, c'est-à-dire des lignes qui ont pour but de donner des indications sur les instructions effectuées et d'expliquer le fonctionnement du programme (algorithme) sans que le compilateur ne les prenne en compte.

Dans la suite de ce livre, nous utiliserons trois types de commentaires :

// Commentaire sur une ligne

/* Commentaire */

' Commentaire /* Commentaire sur plusieurs lignes */

Remarque :

Parfois on utilise les commentaires pour annuler l'action de quelques instructions dans un algorithme ou un programme au lieu de les effacer. Voir l'exemple suivant :

Variable i : entier // Variable i : réel
i ← 4 /* i ← 4 i ← 4 * I */
Les instructions précédentes sont équivalentes à : Variable i :

entier

i ← 4

1

Chapitre 02 : Stratégies de résolution des problemes.

Objectif : - Définition de la stratégie de résolution (Contextes, Entrées/Sorties , traitements)

I. **Algorithmique et programmation**

L'algorithmique est la discipline qui s'intéresse aux algorithmes.
Tout problème à programmer doit être résolu, d'abord sous forme d'algorithme, puis converti en programme dans le langage de votre choix. En effet, un algorithme est

1

indépendant du langage de programmation utilisé. Un programme est un enchaînement d'instructions, écrit dans un langage de programmation, exécutées par un ordinateur, permettant de traiter un problème et de renvoyer des résultats. Il représente la traduction d'un algorithme à l'aide d'un langage de programmation. Le cycle de développement d'un programme (ou d'une application) informatique peut se résumer ainsi (figure1) :

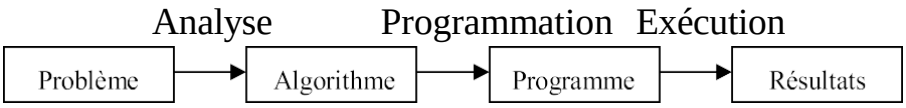


Figure 1 : Cycle de développement d'un programme Exemple :

Parmi les langages de programmations, on peut citer : Pascal, C, C++, Php, Java, C#, Visual Studio etc.

Nous avons montré au début de ce chapitre que des programmes auront besoin d'une structure qui permet d'exécuter un bloc d'instruction selon le résultat d'une condition. Nous avons dit que ce type d'instructions de structuration est appelé : structure alternative.

II Les structures alternatives

a) Forme 1 : si Finsi

Pour que notre algorithme continue à travailler si une condition (simple ou composée) est vérifiée, alors on doit utiliser une structure alternative avec la forme suivante :

Algorithme : Si (condition) alors.....}bloc d'instructions Fin si

La valeur de la condition sera interprétée en True ou False . Si la condition est correcte (évaluée à True) : le bloc d'instructions s'exécuté. Si la condition est évaluée à False, ce bloc d'instructions **sera omis** (ignoré) et la prochaine instruction exécutée sera celle qui suit le 'Fin si' en algorithmique ou le niveau d'indentation d'origine.

Exercice : écrire un algorithme, puis un programme qui demande à l'utilisateur deux nombres et une opération (+ ou –). Puis, effectue l'opération arithmétique demandée et affiche le résultat obtenu.

Exemples d'exécution :

```
Donner deux nombres
15
2
Donner une opération à effectuée:
+
La somme :  17
```

```
Donner deux nombres
6
9
Donner une opération à effectuée:
-
La différence: -3
```

```
Donner deux nombres
14
5
Donner une opération à effectuée:
k
l'opération n'est pas l'addition ni la soustraction
```

b) **Forme 2 : si sinon Finsi** :

Pour pouvoir proposer le traitement à faire même-ci une condition est erronée, on peut utiliser cette forme plus générale de la structure alternative :

Algorithme : **Si (condition) alors ..... }bloc1 d'instructions Sinon .....}bloc2 d'instructions Fin si**

Si la condition mentionnée après if est VRAI (True), on exécute le bloc1 d'instructions; si la condition est fausse, on exécute le bloc2 d'instructions.

Ecrire un algorithme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif ou positif (on laisse de côté le cas oû le produit est nul). Attention toutefois : on ne doit pas calculer le produit des deux nombres.

Algorithme Signe Produit

Var : m, n : entier

Début

Ecrire (“Entrez le premier nombre”)

Lire (m)

Ecrire (“Entrez le second nombre”)

Lire (n)

Si (m > 0 ET n > 0) OU (m < 0 ET n < 0)

Alors Ecrire (“Leur produit est positif”)

Sinon Ecrire (“Leur produit est négatif”)

Fin Si

Fin

c) Forme 3 : Les structures alternatives imbriquées

Dans ce cas, la structure alternative contient à son tour une structure alternative: on dit qu’on a des structures alternatives imbriquées les unes dans les autres.

Syntaxe : si(condition1) alors si(condition2) alors .... sinon ..... fin si fin si

Exercice :

Ecrire un algorithme qui demande l’age d’un enfant `a l’utilisateur. Ensuite, il ´ l’informe de sa catégorie : “Poussin” de 6 `a 7 ans “Pupille” de 8 `a 9 ans “Minime” de 10 `a 11 ans “Cadet” apr`es 12 ans.

Algorithme Age Enfant
Var : age : entier
Début
Ecrire (“Entrez l’age de l’enfant”)

Lire (age)
Si age >= 12 Alors
Ecrire (“Cat´egorie Cadet”)
Sinon Si age >= 10
Alors Ecrire (“Cat´egorie Minime”)
Sinon Si age >= 8 Alors
Ecrire (“Catégorie Pupille”)
Sinon Si age >= 6 Alors
Ecrire (“Cat´egorie Poussin”)
Fin Si
Fin

d) **Forme 4 : Cas où**

Dans plusieurs langages de programmation, la structure alternative peut prendre une autre forme qui permet d’imbriquer plusieurs cas. Sa syntaxe est :

Algorithme **cas où v vaut v1 : bloc1d'instructions v2 : bloc 2 d'instructions . . . vn : bloc n d'instructions fincas**

III Les boucles (les structures itératives)

L’utilité : C’est une structure de l’algorithmique qui permet de résoudre des problèmes en répétant un traitement plusieurs fois pour une même donnée.

Par exemple : Ecrire un algorithme qui permet d’afficher le mot ‘’bonjour’’ 50 fois.
50 : nombre de répétition. Le traitement à répéter le mot ‘’Bonjour’’.

Remarque : on dit le nombre de répétition ou le nombre d’itérations (qui signifié le passage d’un pas à l’autre dans une boucle).

- On distingue trois types de boucles : La structure pour, la structure tant que et la structure répéter jusqu'à.

a- La structure Pour :

Propriétés : On utilise la structure pour quand le nombre d'itération est connu à l'avance.

- Le compteur est initialisé à 1.
- Le pas d'incrémentation =1

Syntaxe :

Pour compteur allant de (valeur initiale) à (valeur finale) faire

∑ Instructions

Fin pour

Exemple :

Pour i allant de 1 à 3 faire

Ecrire (''bonjour'')

Fin pour

b- La structure Tant que :

Contrairement à la boucle pour, la structure tant que permettent de faire des itérations tant que la condition est vérifiée.

Propriétés :
Le nombre d'itérations n'est pas connu à l'avance,
Permet de vérifier si la condition est vraie pour exécuter le bloc d'instructions, si la condition est fausse on sort de la boucle.

Syntaxe : Tant que (condition)
faire ∑ Instructions
Fin tant que

Exemple : S ←0 Tant que (i<=5)
faire S ←S+i i ←i+1
Fin tant que
Ecrire(s) Cet algorithme s'arrête des que le compteur i>5

Remarque : Si on connait le nombre de répétition à traiter on utilise la boucle pour mais si on connait la condition mais on ne connait pas le nombre de répétition on utilise la boucle tant que.

b- La structure répéter jusqu'à:

Elle est semblable à la structure de tant que mais la différence est que la boucle tant que permet d'exécuter le bloc d'instructions tant que la condition est vraie contrairement à la boucle répéter jusqu'à qui permet d'exécuter le bloc d'instruction jusqu'à la condition devient vraie.

Remarque : La boucle tant que vérifie la condition avant chaque itérations (au début) mais La boucle répéter jusqu'à vérifie la condition après chaque itération (à la fin).

Syntaxe :
Répéter ∑ Instructions Jusqu'à
(condition sera vraie)

Exemple :
S←0
répéter S←S+i
i← i+1 Jusqu'a
(i>5)
Fin répéter
Ecrire(s)

Chapitre 03 : Calcul de Complexité d'un algorithme.

INTRODUCTION

Un algorithme est une suite d'instructions qui décrit comment résoudre un problème particulier en un temps fini. Avez-vous déjà ouvert un livre de recettes de cuisine ? ou déjà indiqué un chemin à un touriste?
Si oui, sans le savoir, vous avez déjà exécuté des algorithmes.

Si l'algorithme est juste, le résultat est le résultat voulu, et le touriste se retrouve là où il voulait aller. Si l'algorithme est faux, le résultat est aléatoire, et le touriste est perdu.

Pour fonctionner, un algorithme doit contenir uniquement des instructions Compréhensibles par celui qui devra l'exécuter

Un programme (code) : la réalisation (l'implémentation) d'un algorithme au moyen d'un langage donné (sur une architecture donnée). Il s'agit de la mise en œuvre du principe.

Exemple de tâche: Décider si un tableau L est trié en ordre croissant. ¬ Raisonnement: Un tableau L est trié si tous ses éléments sont dans l'ordre croissant. ¬ Algorithme: Une

fonction vérifiant cette propriété, supposera donc le tableau L, de taille n, trié au départ et cherchera une contradiction.

¬ Code:

```
Fonction trie (L: tab, n: entier) : booleen
Variables i, n: entier; Ok: booleen
Début
Ok ← vrai
pour i de 1 à n faire
si (L[i ] > L[i +1]) alors
 Ok ← Faux
Finsi
Finpour
Retourner OK
Fin trie
 boolean trie (tab L, int n)
{ Ok =true; For (i =0; i< n ; i++) { if (L[i ] > L[i +1]) Ok= Faux; }
return Ok;}
```

La question la plus fréquente qui se pose à chaque programmeur est la suivante: Comment choisir parmi les différentes approches pour résoudre un problème? Exemples: Trier un tableau: algorithme de tri par insertion ou de tri rapide?…, etc

On attend d'un algorithme qu'il résolve correctement et de manière efficace le problème à résoudre, quelles que soient les données à traiter.

1. La correction: résout-il bien le problème donné? Trouver une méthode de résolution (exacte ou approchée) du problème.
2. L'efficacité: en combien de temps et avec quelles ressources? Il est souhaitable que nos solutions ne soient pas lentes, ne prennent pas de l'espace mémoire considerable.

L'efficacité d'un algorithme peut être évaluée par:

- Rapidité (en terme de temps d'exécution)
- Consommation de ressources (espace de stockage, mémoire utilisée)

¬ La théorie du calcul de la complexité étudie l'efficacité des algorithmes. ⌋On s'intéresse dans cette partie, essentiellement, à l'efficacité en terme de temps d'exécution.

THEORIE DE LA COMPLEXITE : définition et objectifs

 La théorie de la complexité est une branche de l'informatique théorique, elle cherche à calculer, formellement, la complexité algorithmique nécessaire pour résoudre un problème P au moyen de l'exécution d'un algorithme A.

La théorie de la complexité vise à répondre aux besoins d'efficacité des algorithmes (programmes):

Elle permet:

• Classer les problèmes selon leur difficulté.

• Classer les algorithmes selon leur efficacité.

• Comparer les algorithmes résolvant un problème donné afin de faire un choix sans devoir les implementer.

1- Evaluation de la Rapidité d'un algorithme

 On ne mesure pas la durée en heure, minute, seconde… cela impliquerait d'implémenter les algorithmes qu'on veut comparer.

Ces mesures ne seraient pas pertinentes car le même algorithme sera plus rapide sur une machine plus puissante.

Au lieu de ça, on utilise des unités de temps abstraite proportionnelles au nombre d'opérations effectuées.

Au besoin, on pourra alors adapter ces quantités en fonction de la machine sur laquelle l'algorithme s'exécute.

2- Calcul du temps de Complexité

Règles:

• Chaque instruction basique consomme une unité de temps (affectation d'une variable, lecture, écriture, comparaison,…).

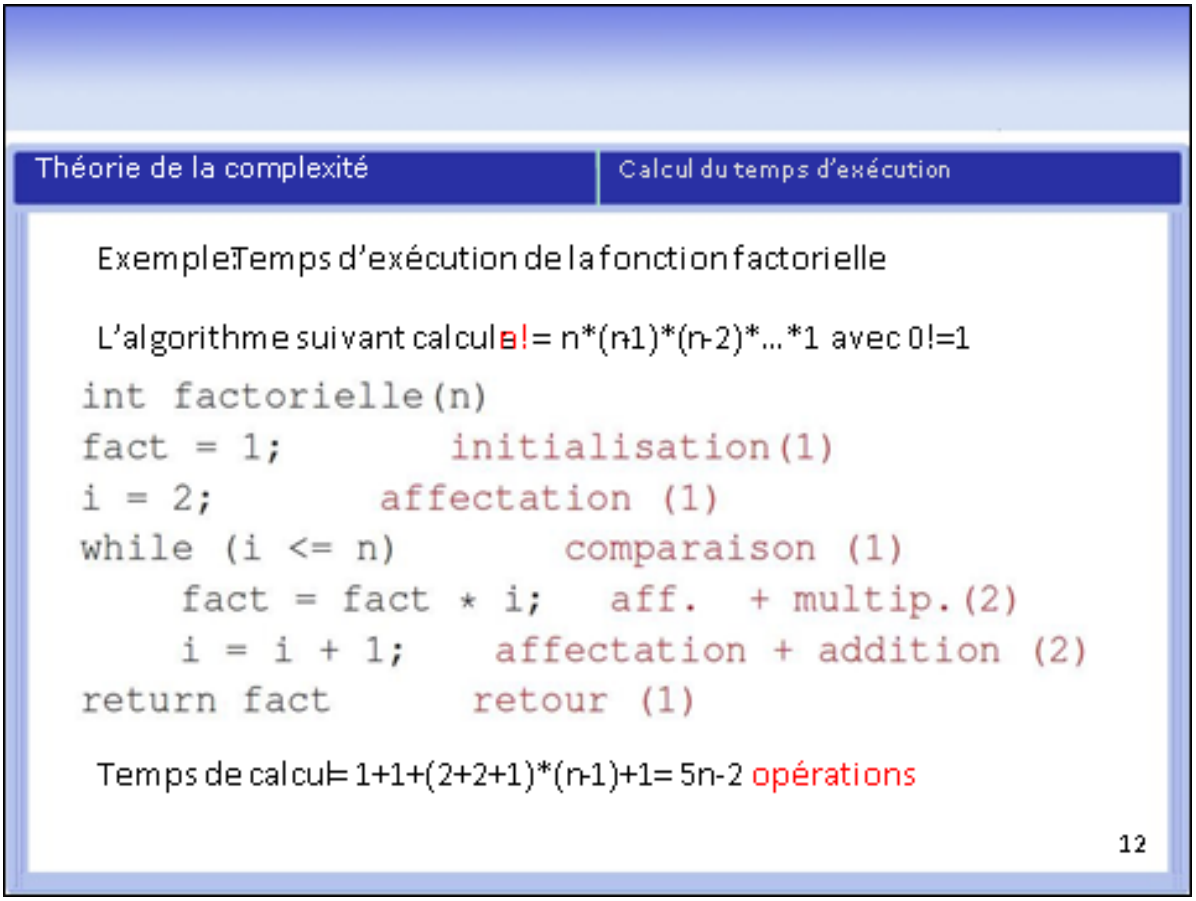
• Chaque itération d'une boucle rajoute le nombre d'unités de temps consommés dans le corps de cette boucle.

• Chaque appel de fonction rajoute le nombre d'unités de temps consommées dans cette fonction.

• Pour avoir le nombre d'opération effectuées par l'algorithme, on additionne le tout.

Exemple: Temps d'exécution de la fonction factorielle

L'algorithme suivant calcule : n! = n*(n-1)*(n-2)*…*1 avec 0! =1



Temps de calcul = 1+1+(2+2+1)*(n-1)+1= 5n-2 opérations

Problèmes

Unités de temps abstraites:

- **Dépends des données.**

- **Dépend de la nature des données: on ne sait pas toujours combien de fois exactement on va exécuter une boucle.**

- **De même lors d'un branchement conditionnel, le nombre de comparaisons effectués n'est pas toujours le même.**

Temps exacte:

- **Dépend de la puissance de la machine.**

- **Dépend de la nature des données (variables): si on change les données, le temps change.**

Solution

Calcul de la complexité algorithmique

CALCUL DE LA COMPLEXITE ALGORITHMIQUE

DEFINITION

La complexité d'un algorithme est la mesure du nombre d'opérations fondamentales qu'il effectue sur un jeu de données. Elle est exprimée comme une fonction de la taille du jeu de données.

Il existe trois types de complexité:

- **Complexité au meilleur**

C'est le plus petit nombre d'opérations qu'aura à exécuter l'algorithme sur un jeu de données de taille n.

- **Complexité au pire**

C'est le plus grand nombre d'opérations qu'aura à exécuter l'algorithme sur un jeu de données de taille n.

Complexité en moyenne

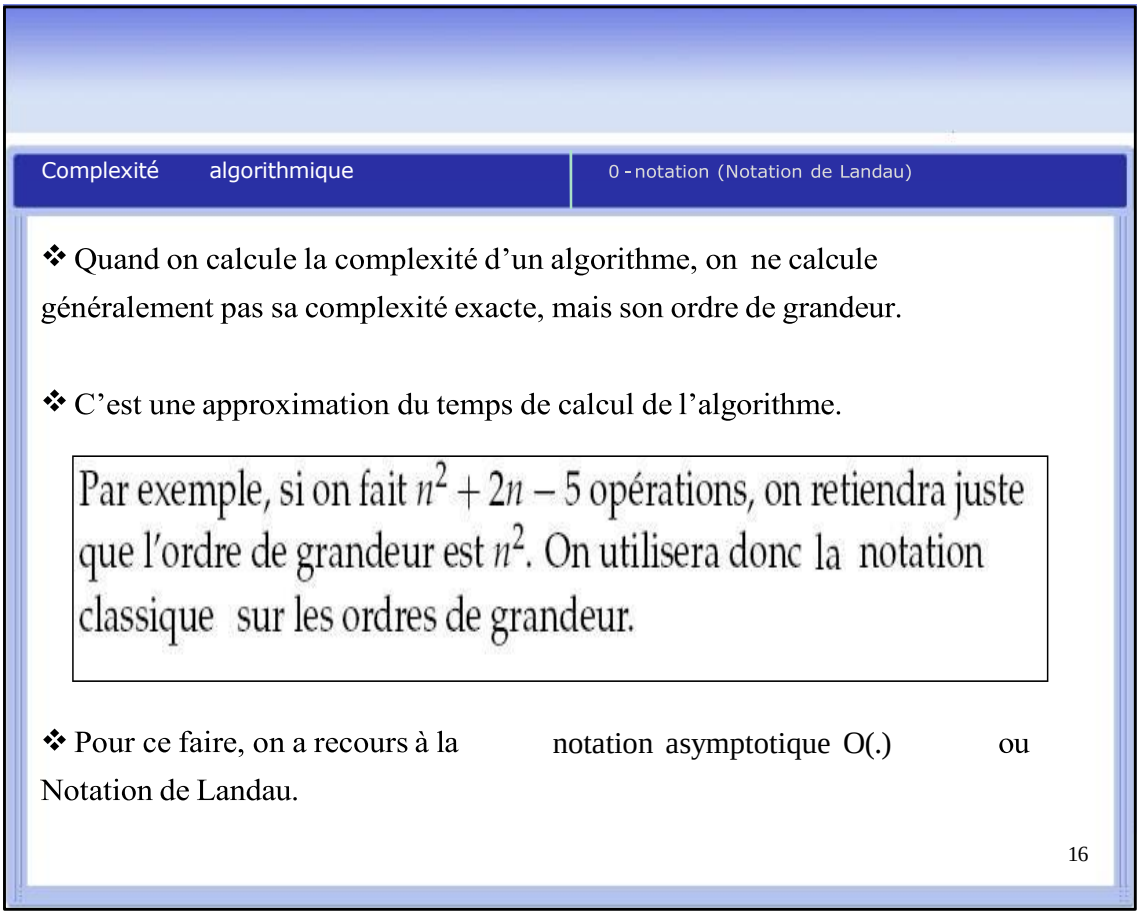
C'est la moyenne des complexités de l'algorithme sur des jeux de données de taille n.

De façon générale: La complexité d'un algorithme est une mesure de sa performance asymptotique dans le pire des cas.

- asymptotique ?
  on s'intéresse à des données très grandes parce que les petites valeurs ne sont pas assez informatives.

- Pire des cas ?
  on s'intéresse à la performance de l'algorithme dans les situations où le problème prend le plus de temps à résoudre parce qu'on veut être sûr que l'algorithme ne prendra jamais plus de temps que ce qu'on a estimé.

Un algorithme est dit « optimal » si sa complexité est la complexité minimale parmi les algorithmes de sa classe.



Cette notation exprime la limite supérieure d'une fonction dans un facteur constant.

⎫ Soit n la taille des données à traiter, on dit qu'une fonction f(n) et en O(g(n)) si :

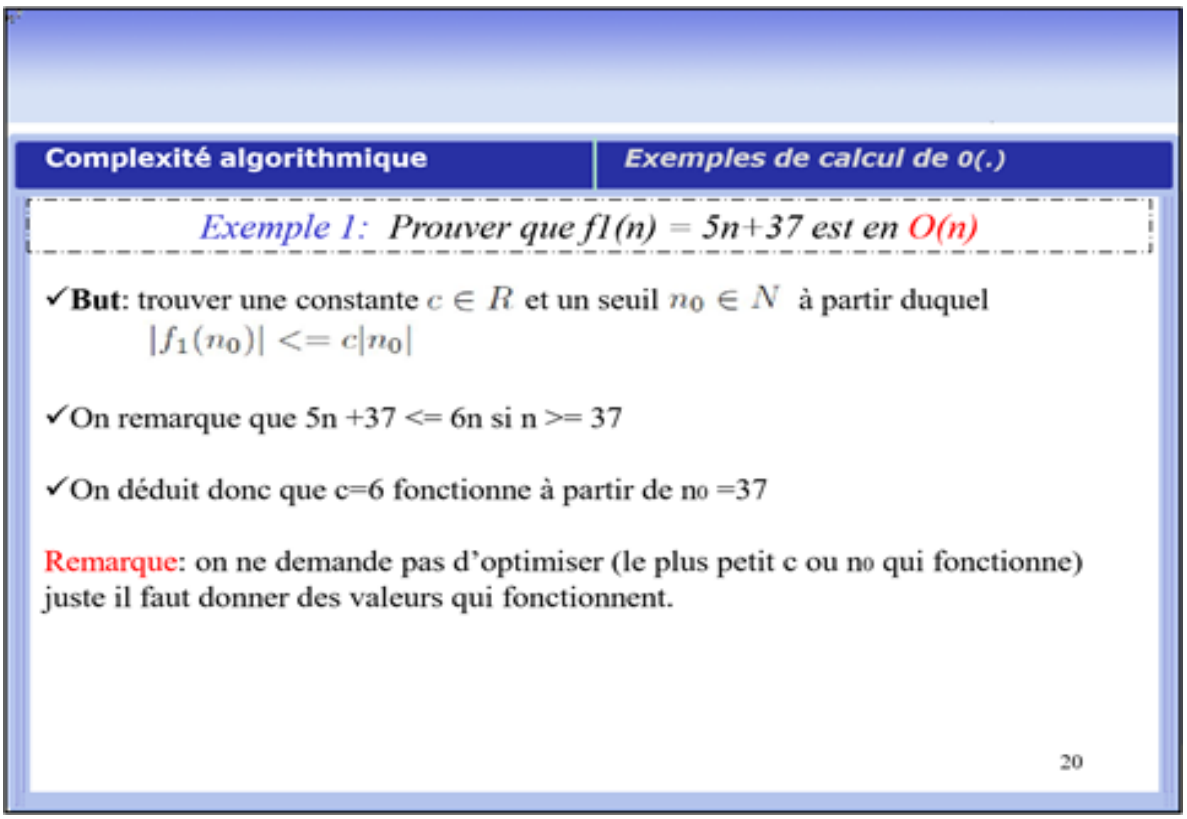
$$\exists n_0 \in N, \quad \exists c \in R, \quad \forall n >= n_0 : \quad |f(n)| <= c|g(n)|$$

⎫ f(n) et en O(g(n))s'il existe un seuil à partir duquel la fonction f(.) est toujours dominée par g(.), à une constante multiplicative fixée près.

Exemple: $T(n) = O(n^2)$ veut dire qu'il existe une constante $c > 0$ et une constante $n_0 > 0$ tel que pour tout $n > n_0$ $T(n) <= c\ n^2$

Utilité: **Le temps d'exécution est borné**
Signification: **Pour toutes les grandes entrées (i.e., N >= n0),** on est assuré que l'algorithme ne prend pas plus de c*g(n) étapes.

CALCUL DE COMPLEXITE :



Exemple 2: Prouver que $f_2(n) = 6n^2 + 2n - 8$ *est en* $O(n^2)$

✓**But**: trouver une constante $c \in R$ et un seuil $n_0 \in N$ à partir duquel

$$|6n_0^2 + 2n_0 - 8| <= c|n_0^2|$$

✓Chercher d'abord c, c=6 ne peut pas marcher, donc c=7

✓On doit donc trouver un seuil n_0 à partir du quel :

$$|6n^2 + 2n - 8| <= 7|n^2| \quad \forall n >= n_0$$

✓c=7 et n_0 =1

Exemple 3: Calculer la complexité de $\mathbf{T}(n) = c_1n^2 + c_2n$

✓On remarque que $c_1n^2 + c_2n <= c_1n^2 + c_2n^2 = (c_1 + c_2)n^2$
pour tout n>= 1

✓ $\mathbf{T}(n) <= cn^2$ où $c = c_1 + c_2$ et $n_0 = 1$

✓Donc $\mathbf{T}(n)$ est en $O(n^2)$

Exemple 4: Calculer la complexité de l'initialisation d'un tableau:

```
for (int i=0; i<n; i++) Tab[i]=0;
```

✓On remarque qu'il y a n itérations; chaque itération nécessite un temps d'exécution <= c où c est une constante (accès au tableau + une affectation)

✓ le temps est donc T(n)<= c n

✓Donc T(n) est en O(n)

On calcule le temps d'exécution, mais on effectue les simplifications suivantes:

- On oublie les constantes multiplicatives (elles valent 1)
- On annule les constantes additives
- On ne retient que les termes dominants

Exemple (simplifications) soit $g(n) = 4n^3 - 5n^2 + 2n + 3$:

- On remplace les constantes multiplicatives par 1: $1n^3 - 1n^2 + 1n + 3$
- On annule les constantes additives : $n^3 - n^2 + n + 0$
- On garde le terme de plus haut degré: $n^3 + 0$
- On a donc la complexité de $g(n) = O(n^3)$

De façon générale, les règles de la notation en O sont les suivantes:

- Les termes constants :

$$O(c) = O(1)$$

- Les constantes multiplicatives sont omises:

$$O(cT) = c\,O(T) = O(T)$$

- L'addition est réalisée en prenant le maximum:

$$O(T1)+O(T2) = O(T1+T2) = \max(O(T1); O(T2))$$

- La multiplication reste inchangée:

$$O(T1)\,O(T2) = O(T1*T2)$$

❑ Exemple:

Supposons que le temps d'exécution d'un algorithme est décrit par la fonction:

$T(n) = 3n^2+10n+10$, calculer $O(T(n))$?

$$\begin{aligned} O(T(n)) &= O(3n^2 + 10n + 10) \\ &= O(\max(3n^2, 10n, 10)) \\ &= O(3n^2) \\ &= O(n^2) \end{aligned}$$

❑ Remarque:

Pour n=10, nous avons:

Temps d'exécution de $3n^2$: 3(10)2 / 3(10)2+10(10)+10 = 73,2%

Temps d'exécution de 10n : 10(10) / 3(10)2+10(10)+10 = 24,4%

Temps d'exécution de 10 : 10 / 3(10)2+10(10)+10 = 2,4%

Le poid $3n^2$ peut donc négliger les quantités10n et 10.

✓ Ceci explique les règles de notation O.

Cas d'une instruction simple : Les instructions de base (lecture, écriture, affectation …) prennent un temps constant, noté O(1).

Cas d'une suite d'instructions: le temps d'exécution d'une séquence est déterminé par la règle de la somme.

Traitement1 $T_1(n)$

Traitement2 $T_2(n)$

$$T(n) = T_1(n) + T_2(n)$$

$$O(T) = O(T1 + T2) = \max(O(T1); O(T2))$$

Cas d'une branche conditionnelle: le temps d'exécution est déterminé aussi par la règle de la somme.

```
if <condition>:          O(g(n))
    # instructions (1)   O(f1(n))
else:
    # instructions (2)   O(f2(n))
```

$$= O(g(n) + f_1(n) + f_2(n))$$

➢ Cas d'une boucle : On multiplie la complexité du corps de la boucle par le nombre d'itérations.

Exemple pour la boucle while (tant que), la complexité se calcule comme suit :

```
# en supposant qu'on a m itérations
while <condition>:        O(g(n))
    # instructions        O(f(n))
```

$$\left.\begin{matrix} O(g(n)) \\ O(f(n)) \end{matrix}\right\} = O(m * (g(n) + f(n)))$$

❑ Pour calculer la complexité d'un algorithme:

- On calcule la complexité de chaque partie de l'algorithme.
- On combine ces complexités conformément aux règles déjà vues.
- On effectue sur le résultat les simplifications possibles déjà vues.

Exemple: calcul de la complexité de la fonction factorielle

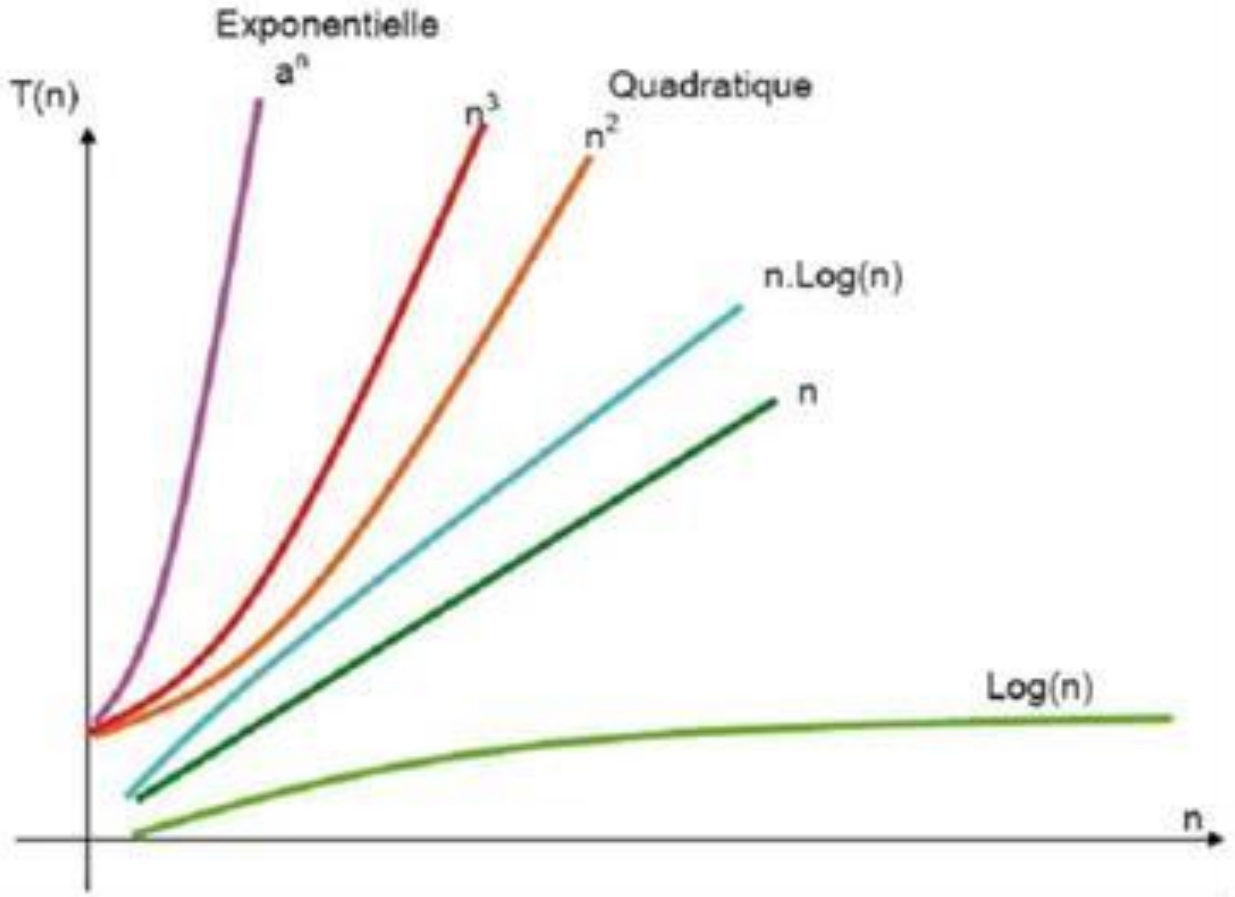
```
int factorielle(n)
fact = 1;              O(1)
i = 2;                 O(1)
while (i <= n)         O(1)
    fact = fact * i;   O(1)      } O(n)
    i = i + 1;         O(1)
return fact            O(1)
```

Complexité de la fonction = O(1) +O(1)+O[(n -1)*(1+1+1)]+O(1)
= O(1)+O(1)+O(n)+O(1)
= O(n)

CLASSE DE COMPLEXITE:

- On peut ranger les fonctions équivalentes dans la même classe.
- Deux algorithmes de la même classe sont considérés de même complexité.
- Les classes de complexité les plus fréquentes (par ordre croissant selon O(.))

Complexité	Classe
$O(1)$	constant
$O(logn)$	logarithmique
$O(n)$	linéaire
$O(nlogn)$	sous-quadratique
$O(n^2)$	quadratique
$O(n^3)$	cubique
$O(2^n)$	exponentiel
$O(n!)$	factorielle



Exemples de temps d'exécution en fonction de la taille de la donnée et de la complexité de l'algorithme, si on suppose qu'une instruction est de l'ordre de la μs;

T.\C.	$\log n$	n	$n \log n$	n^2	2^n
10	$3\mu s$	$10\mu s$	$30\mu s$	$100\mu s$	$1000\mu s$
100	$7\mu s$	$100\mu s$	$700\mu s$	$1/100s$	10^{14} *siècles*
1000	$10\mu s$	$1000\mu s$	$1/100s$	$1s$	*astronomique*
10000	$13\mu s$	$1/100s$	$1/7s$	$1,7mn$	*astronomique*
100000	$17\mu s$	$1/10s$	$2s$	$2,8h$	*astronomique*

Exemple Pratique :

Ecrire un algorithme qui permet de calculer le produit C de deux matrices carré A et B de dimension (nxn) ensuite de déterminer sa complexité?

Principe:

➢ L'équation C=A *B s'écrit:

$$\begin{pmatrix} r & s \\ t & u \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} e & g \\ f & h \end{pmatrix}$$

C A B

➢ En développant cette équation, nous obtenons :

$$r = ae + bf, \quad s = ag + bh, \quad t = ce + df \quad \text{et} \quad u = cg + dh.$$

Application de calcul de complexité multiplication de deux matrices

Ecrire un algorithme qui permet de calculer le produit C de deux matrices carré A et B de dimension (nxn) ensuite de déterminer sa complexité?

Algorithme:

```
Procédure MULTI -MAT (var C:mat; A, B: mat, n:entier )
Var i, j, k: entier
Début
Pour i de 1 à n faire
    Pour j de 1 à n faire
        C[ i,j ] □ 0
        Pour k de 1 à n faire
            C[ i,j ] □ C[ i,j ] +A [i,k ] * B [k,j ]
Finpour finpour finpour
Fin
```

Ecrire un algorithme qui permet de calculer le produit C de deux matrices carré A et B de dimension (nxn) ensuite de déterminer sa complexité?

Complexité:

Procédure MULTI-MAT(var C:mat; A, B: mat, n:entier)
Var i, j, k: entier
Début

Pour i de 1 à n faire
  Pour j de 1 à n faire
    C[i,j] ← 0 O(1)
    Pour k de 1 à n faire
      C[i,j] ← C[i,j] +A[i,k] * B[k,j] O(1)
  Finpour finpour finpour
Fin

Nbre d'iter: n
=>n*O(n)= O(n*n)

Nbre d'iter: n
=>n*O(n*n)= O(n*n*n)

Nbre d'iter: n
=> O(n)

$$O(T)=O(n^3)$$

35

Chapitre 04 : Analyse de l'algorithme de tri.

Auparavant vous avez dû voir que la recherche d'un élément dans un tableau était plus rapide si ce tableau était ordonné. Il est donc naturel de se demander s'il existe une procédure efficace pour trier des données. Nous allons observer différents algorithmes de tri et surtout comparer leurs complexités respectives afin de montrer qu'ils ne sont pas équivalents.

On suppose que les éléments trier sont des entiers (mais les algorithmes proposés sont valables pour n'importe quel type d'éléments muni d'un ordre total). On suppose qu'on trie des tableaux par ordre croissant. On note N le nombre d'éléments triés.

1 TRI PAR SELECTION

C'est le tri dit naïf. Il consiste recherche le minimum de la liste, et le placer en début de liste puis recommencer sur la suite du tableau.

exemple sur T=[4,12,5,8,9,6,13,3] :

Etape 0 : on cherche le minimum sur T[0 :8] et on l'échange avec T[0]	4	12	5	8	9	6	13	3
Etape 1 : on cherche le minimum sur T[1 :8] et on l'échange avec ... T[1].	3	12	5	8	9	6	13	4
Etape 2 :…….	3	4	5	8	9	6	13	12
…								

Ecrivons l'algorithme :

```
def imin(T,a,b): #fonction qui renvoie la position de la valeur minimale de T
#entre les positions a et b
    imin=a
    N=len(T)
    for i in range(a+1,b):
        if T[i] < T[imin]:
            imin = i
    return imin

def triselection(T):
    N=len(T)
    for i in range(N-1):
        j=imin(T,i,N)
        T[i],T[j] = T[j],T[i]
    return T
```

Complexités :

Quelle que soit l'étape, on n'a pas besoin d'un autre tableau donc le coût en mémoire de ce tri est constant.

Concernant sa complexité temporelle :

- en comparaisons : pour rechercher le premier minimum on e ectue N-1 comparaisons, pour le second N-2, ... le nombre total de comparaisons est de $(N-1)+(N-2)+\ldots+1=\frac{N(N-1)}{2}$ donc en $O(N^2)$
- en affectations : pour chaque échange, on effectue 3 affectations donc 3N, pour chaque recherche de minimum, au maximum N-1, puis N-2, ... donc on retrouve une complexitØ en $O(N^2)$.

Quel est le pire tableau trier avec cette méthode?

2. TRI A BULLES

Nom anglais : bubble sort - Propriétés : tri interne, non stable, sur place
Complexité dans tous les cas en Θ (n^2)
Le principe de cet algorithme est d'échanger les clés contiguës qui ne sont pas correctement triées. Son nom vient du fait que les clés se déplacent comme des bulles dans une flûte de champagne. Elles remontent d'ailleurs si lentement que cela explique la complexité quadratique.

Algorithme issu TRI-BULLES(A)

```
1 pour i ← 1 à n faire
2 pour j ← n à i + 1 faire
3 si A[j] < A[j - 1] alors
4 PERMUTER(A, j, j - 1)
5 fin si
6 fin pour
7 fin pour
```

3 TRI PAR INSERTION

C'est le tri du joueur de cartes. Il consiste insérer successivement chaque élément T[i] dans la portion du tableau T[0 :i] déjà triée.
exemple sur T=[4,12,5,8,9,6,13,3]. Le tableau T[0 :1]=[4] est déjà triée.

Etape 1 : on cherche placer T[1]=12 dans T[0 :1]=[4]	4	12	5	8	9	6	13	3
Etape 2 : on cherche placer T[2]=5 dans T[0 :2]=[4,12]	4	5	12	8	9	6	13	3
Etape 3 :								
...								

Ecrivons l'algorithme :

```
def triinsertion(T):
    n=len(T)
    for i in range (1,n):
        valeur=T[i] #valeur a inserer
        j=i-1
        while (j>=0) and (T[j]>valeur):
            T[j+1]=T[j]
            j=j-1
        T[j+1]=valeur
```

Complexités :

Quel est le pire tableau trier avec cette méthode? le meilleur?

Une fois de plus, le coût en mémoire de ce tri est constant.

Evaluons sa complexité temporelle :
pour le premier placement, on effectue 2 comparaisons;

pour le second placement, on effectue au mieux 2 comparaisons (si le tableau est déjà trié), au pire 4 comparaisons
...
pour le dernier placement, on effectue au mieux 2 comparaisons (si le tableau est déjà trié), au pire 2(N-1) comparaisons

Le nombre total de comparaisons est donc de :
au mieux de $2(N-1)$ comparaisons donc $O(N)$ temps linéaire si le tableau est trié.
au pire : $2(1+2+\ldots+N-1) = 2\frac{N(N-1)}{2}$ donc en $O(N^2)$, temps quadratique si le tableau est rangé par ordre décroissant. en moyenne $N^2/2$.

Le nombre d'affectations peut être un peu optimisé. Il sera au mieux égal au nombre de comparaisons donc quadratique.

4 TRI FUSION (TD)

Comme le tri rapide,le tri fusion applique le principe du diviser pour mieux régner.
Il partage arbitrairement les éléments trier en deux sous ensembles de même taille (sans chercher à les comparer) afin d'éviter le pire cas du tri rapide oû les deux sous ensembles sont de tailles disproportionnées. Une fois les deux parties triées récursivement, il les fusionne.

exemple sur T=[4,12,8,5,9,6,13,3].

pour trier T[0 :8] on trie T[0 :4] et T[4 :8]	4	12	8	5	9	6	13	3
pour trier T[0 :4] on trie T[0 :2] et T[2 :4]	4	12	8	5				
pour trier T[0 :2] on trie T[0 :1] et T[1 :2]	4	12						
on fusionne T[0 :1] et T[1 :2] triés								
on fusionne T[0 :2] et T[2 :4] triés								
on fusionne T[0 :4] et T[4 :8] triés								

Implémentons cette méthode de tri sous Python :

Nous allons utiliser deux fonctions :

- une fonction fusion qui prend en entre deux tableaux T1 et T2 supposés triés et renvoie un tableau contenant les mêmes éléments que T1 et T2 rangés par ordre croissant.
- la fonction récursive de tri qui si le tableau contient plus d'un élément le subdivise en deux sous tableaux de tailles équivalentes et les trie.

```
def fusion(T1,T2):
    n1=len(T1)
    n2=len(T2)
    T=[]
    i=0
    j=0
    while(i<n1 or j<n2):
        if i==n1:
            T.append(T2[j])
            j=j+1
        elif j==n2:
            T.append(T1[i])
            i=i+1
        elif T1[i] < T2[j]:
            T.append(T1[i])
            i=i+1
        else:
            T.append(T2[j])
            j=j+1
    return T

def trirec(T):
    if len(T)<=1:
        return T
    else:
        m=len(T)//2
        T=fusion(trirec(T[0:m]),trirec(T[m:len(T)]))
    return T
```

Etudions la complexité :

Soit $C(N)$ le nombre de comparaisons e effectuées par la fonction tri sur un tableau de taille N. $C(N) = 2C(\frac{N}{2}) + f(N)$ où $f(N)$ désigne le nombre de comparaisons effectuées par fusion.

Cas le plus favorable : tous les éléments d'un des tableaux sont inférieurs tous les éléments de l'autre sous tableau. On n'effectue vraiment que $f(N) \approx \frac{N}{2}$ comparaisons. On a alors $C(N) \approx \frac{1}{2} N \log_2(N)$.

Cas le moins favorable : il faut examiner tous les éléments, soit $f(N) \approx N - 1$ comparaisons. On a alors $C(N) \approx N \log_2(N)$.

Concernant le nombre $A(N)$ d'affectations, $A(N) = 2A(N/2)+2N$ dans tous les cas donc $A(N) \approx 2N \log_2(N)$.

Conclusion : dans tous les cas, la complexité en comparaison et en affectation est en $N \log_2(N)$, qui est la complexité optimale du tri a priori .

remarque : Pourquoi la complexité en comparaisons d'un algorithme de tri est elle au mieux en $N \log_2(N)$?

On peut visualiser notre problème de tri comme un arbre binaire, chaque noeud représentant une comparaison à effectuer (on part gauche si la comparaison est positive et droite sinon).

S'il y a N éléments, il y a N! comparaisons possibles, donc notre arbre a N! feuilles.

Une propriété sur les arbres affirme que la hauteur/profondeur d'un arbre N! feuilles est au moins de $\log_2 N!$.

5 TRI RAPIDE

Le tri rapide s'appuie sur le principe diviser pour mieux régner .

On choisit un élément appelé pivot. On sépare les éléments trier en deux sous ensembles:

- le premier constitué de tous les éléments plus petits que le pivot,
- le second constitué des éléments plus grands que le pivot.
- Puis on trie récursivement chaque sous ensemble. Le tri rapide d'un tableau s'effectue en place .

exemple sur T=[4,12,5,8,9,6,13,3] par la mØthode du Tri rapide

L'un des enjeux est le choix du pivot : idéalement il est une médiane des données (il sépare ainsi en deux sous ensembles de même taille). Malheureusement, il est impossible de le savoir l'avance puisque le tableau n'est pas trié. On le choisit donc au hasard!

Implémentons cette méthode de tri sous Python :

Une fois le pivot choisi (la fonction randint(a,b) de la bibliothèque random renvoie un entier compris entre a et b inclus), on a besoin de deux fonctions :
une fonction de partition qui organise les éléments du tableau trier autour du pivot et donne la position nale du pivot. Cette fonction nécessitant de procéder de nombreux échanges de valeurs du tableau, il peut être utile de créer une fonction echange(T,i,j) permettant d'intervertir les contenus des cases i et j du tableau T. • une fonction de tri qui trie récursivement les deux portions de tableau gauche et droite du pivot (d'oû l'importance de connaitre sa position).

```
from random import randint

def echange(T,i,j):
    T[i],T[j]=T[j],T[i]

def partition(T,g,d):

    #choix du pivot
    m=randint(g,d-1)
    p=T[m]
    #on place le pivot à gauche au depart
    echange(T,g,m)
    m=g

    for i in range (g+1,d): #on considere tous les elements
        if T[i]<p:
            echange(T,i,m+1)
            m=m+1

    echange(T,g,m)#on met le pivot a sa place
    return m

def tri(T,g,d):

    if g<d-1: #si le tableau contient au moins 2 elements
        m=partition(T,g,d)
        tri(T,g,m)
        tri(T,m+1,d)


T=[5,6,2,4,8,3,1,7,12,5,1,3,6,4,2,8,9]
tri(T,0,len(T))
print(T)
```

Etude de la complexité en comparaisons :

On note $C(N)$ le nombre de comparaisons pour trier un tableau de taille N.
L'essentiel des comparaisons vient de la fonction partition qui effectue $d - 1 - g$ comparaisons chaque appel.
C'est- -dire au départ N-1 comparaisons.
Ensuite si le pivot a été bien choisi, il a coupé les données en deux sous ensembles de taille équivalente $\frac{N}{2}$. Si le pivot a été mal choisi (si c'est le minimum ou le maximum), on a coupé en un sous ensemble de taille 0 (pas d'élément plus petit ou plus grand que le pivot) et un autre de taille N-1.

• dans le meilleur cas :

$C(N) = N - 1 + 2C(N/2)$ or $C(N/2) = N/2 - 1 +$
$2C(N/4)$ donc $C(N) \approx 2N + 4C(N/4)$
en continuant, $C(N) \approx nN + 2^n C(N/2^n)$ jusqu' ce que $N/2^n$ fasse 1 ou moins. A ce moment l, le sous tableau est déjà trié car il ne contient qu'un élément maximum (complexité nulle).

$$\frac{N}{2^n} \geqslant 1 \Leftrightarrow N \geqslant 2^n \Leftrightarrow \ln(N) \geqslant n \ln(2) \Leftrightarrow n \leqslant \log_2(N)$$

donc $C(N) \approx N \log_2(N)$.

• dans le pire cas :

$C(N) = N - 1 + C(N - 1) = N - 1 + N - 2 + C(N$
$- 2)$... donc $C(N) = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$
La complexité est quadratique.

donc $C(N) \approx N \log_2(N)$.

• en moyenne : On admet que la complexitØ moyenne est en $2N \log_2(N)$.

Etudions la comparaison en affectations :

dans la fonction Tri elle mŒme il y a au plus une affectation. L'essentiel des affectations se produit dans la fonction Partition.

Les deux premiŁres a ectations peuvent être supprimées a n d'optimiser le code en terme d'a ectation mais on ne peut éviter :

- de faire le premier échange pour mettre le pivot gauche (2 a ectations)
- de procéder un échange chaque fois qu'une valeur plus grande que le pivot est trouvée, en incrémentant m.
- de placer le pivot à sa place à la fin s'il n'y est pas.

Il y a autant d'échanges que d'incrémentations de m : soit entre 1 et $(d - g)$.

- **dans le meilleur cas** :

Le meilleur cas en terme d'affectation est donc quand il n'y a pas d'incrémentation de m, c'est dire que le pivot reste tout gauche. Sauf que dans ce cas, cela signifie que nous avons pris comme pivot le minimum et donc tres mal partitionné notre tableau (coupé en 0 éléments inférieurs et N-1 éléments supérieurs). Il y a alors seulement l'echange initial soit deux affectations. Mais il va falloir le faire N fois. La complexité est donc linéaire en affectation (de l'ordre de 2N) mais elle est alors quadratique en terme de comparaisons.

- **dans le cas le plus favorable pour les comparaisons :**

Le pivot se situe au milieu. Il y a donc environ $\frac{d-g}{2}$ échanges soit $d - g$ affectations. Au départ $d - g = N$, puis on a deux sous tableaux de taille $\frac{N}{2}$... jusqu' ce que $\frac{N}{2^n}$ soit inférieur

1. On retrouve une complexité en $N \log_2(N)$.

- **dans le pire cas :**

Le pire cas pour les affectations correspondent aux cas où le pivot est situé l'extrème droite (il est la valeur maximum) car toutes les valeurs précédentes auront été échangées soit $d - g$ échanges. De plus, la partition est alors peu efficace (N-1 valeurs inférieures au pivot et 0 supérieure). On a alors une complexité quadratique en affectation et en comparaisons.

- **en moyenne** :

on admet que la complexité moyenne en terme d'affectation est en $2N \log_2(N)$.

Résumons :

Opérations	meilleur cas pour les a ectations	meilleur cas pour les comparaisons	moyenne	pire cas
comparaisons	$N2$	$N\log_2(N)$	$2N\log_2(N)$	$\frac{N2}{2}$
affectations	$2N$	$N\log_2(N)$	$2N\log_2(N)$	$N2$

remarques :

- un cas peut être le pire ou le meilleur, tout dépend du point de vue. Il est donc important de faire attention aux priorités souhaitées.
- Dans les faits, une comparaison est une opération plus complexe et donc plus chronophage pour l'ordinateur.
- Un autre facteur limitant sera également le nombre d'appel une fonction récursive (limité priori 1000 en Python).

Tableau des comparaisons en terme de temps (en s) :

N	1000	10 000	100 000
Tri par selection	0.079	5.82	565.94
Tri Bulle	0.16	14.51	1476
Tri rapide	0.013	0.09	excede le nb de recursions autorisées
Tri fusion	0.011	0.05	0.69

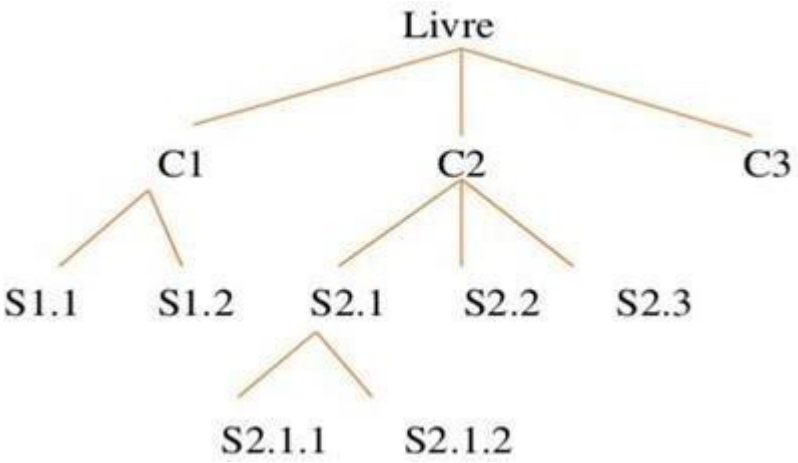
Chapitre 05 : Algorithmique des arbres.

INTRODUCTION

Les arbres sont des structures de données fondamentales en informatique très utilisés dans tous les domaines parce qu'ils sont bien adaptés à la représentation naturelle d'informations homogènes organisées et d'une grande commodité et rapidité de manipulation.

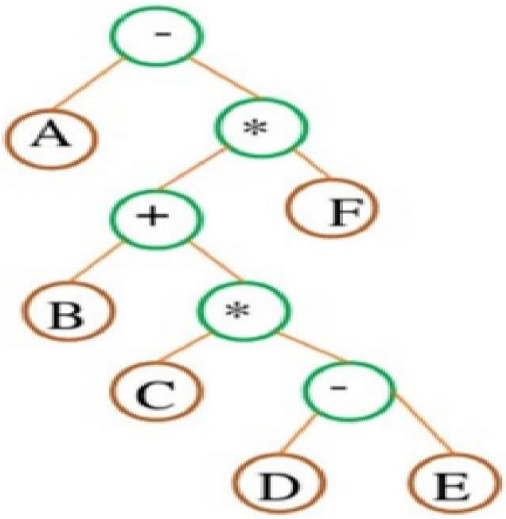
Leur usage est multiple car ils captent l'idée de hiérarchie:

- Découpage d'un livre en chapitres, sections, paragraphes…



- Expression arithmétique:

  L'expression A – (B + C * (D –E)) * F se représente facilement par un arbre où apparait clairement la priorité des opérations:

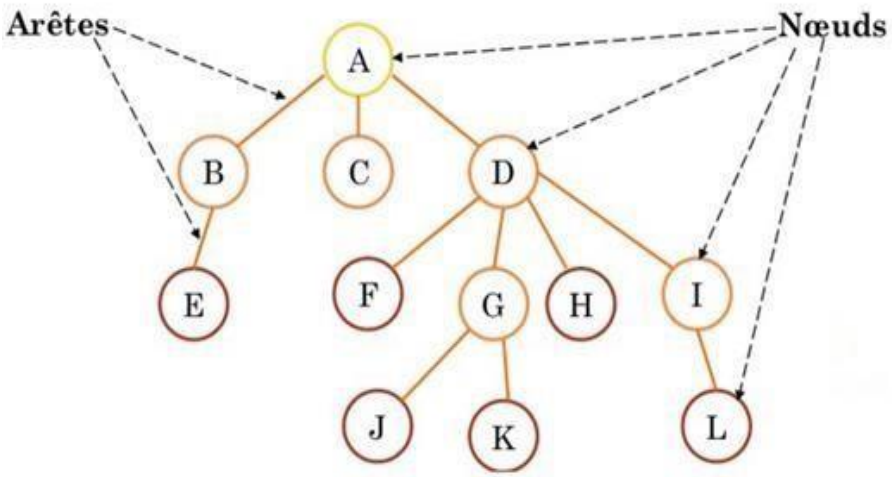


DEFINITION ET TERMINOLOGIES "ARBRE"

- Un arbre est une structure de données (souvent dynamique) représentant un ensemble de valeurs organisées hiérarchiquement (non linéaire).

- Chaque valeur est stockée dans un nœud.

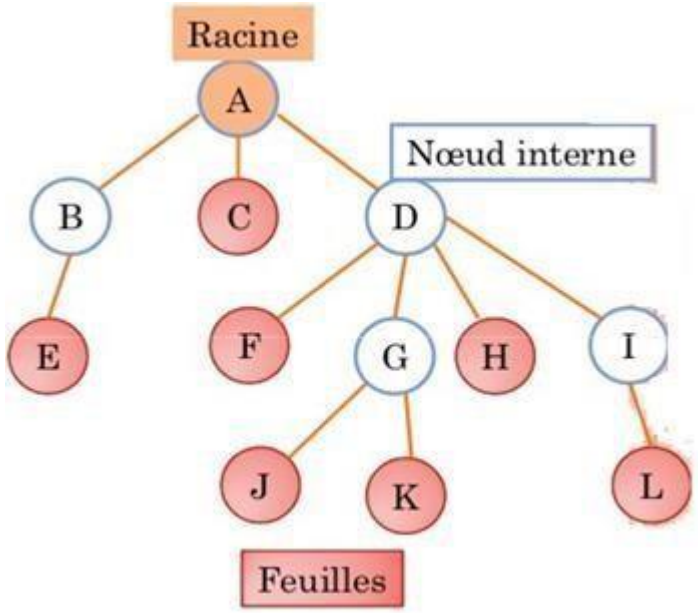
- Les nœuds sont connectées entre eux par des arêtes qui représentent la relation parents/fils.



➢ Racine: c'est le nœud qui n'a pas de prédécesseur (parents) et possède 0 ou plusieurs fils. La racine constitue la caractéristique d'un arbre.

➢ Feuille: c'est un nœud qui n'a pas de successeurs (fils). Une feuille est aussi appelée nœud externe.

➢ Nœud interne: tout nœud qui admet au moins un successeur (fils).

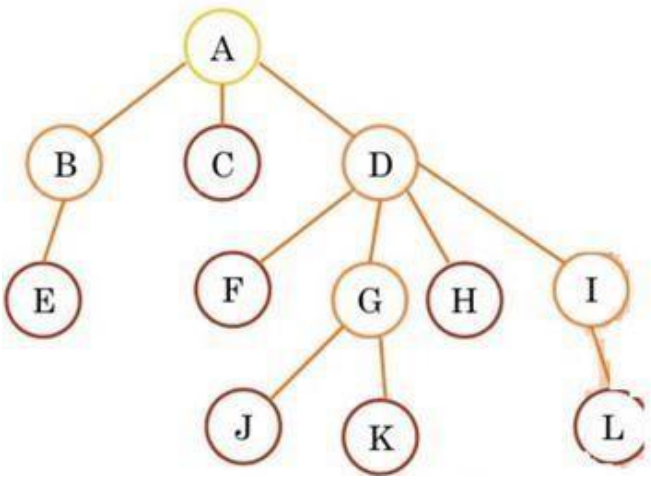


➢ Sous-arbre: est une portion de l'arbre. Dans l'exemple, le nœud G avec ses deux fils J et K constituent un sous-arbre.

➢ Une branche: c'est une suite de nœuds connectées de père en fils (de la racine à la feuille: A-B-E; A-C; A-D-F; A-D-G-J; …

➢ Descendants d'un Nœud: sont tous les nœuds du sous-arbre de racine ce nœud. Dans l'exemple, les descendants de D sont F, G, H, I, J, K et L.

➢ Ascendants d'un nœud: sont tous les nœuds se trouvant sur la branche de la racine vers ce nœud. Dans l'exemple, les ascendants de J sont G, D et A.

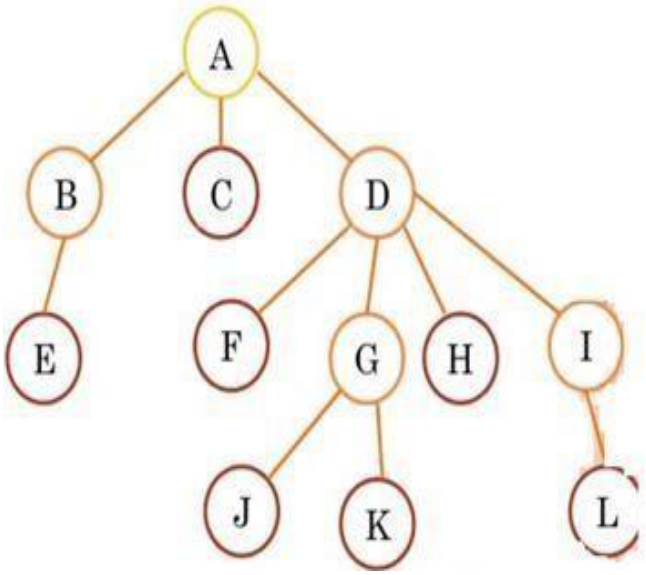


- Taille de l'arbre: est le nombre de nœuds qu'il possède.
  - Taille de l'arbre de l'exemple = 12
  - Taille d'un arbre vide = 0

- Degré d'un nœud: est le nombre de ses fils. Dans l'exemple, le degré de B est 1, le degré de D est 4.

- Degré d'un arbre: est le degré maximum de ses nœuds. Degré de l'arbre de l'exemple est 4.

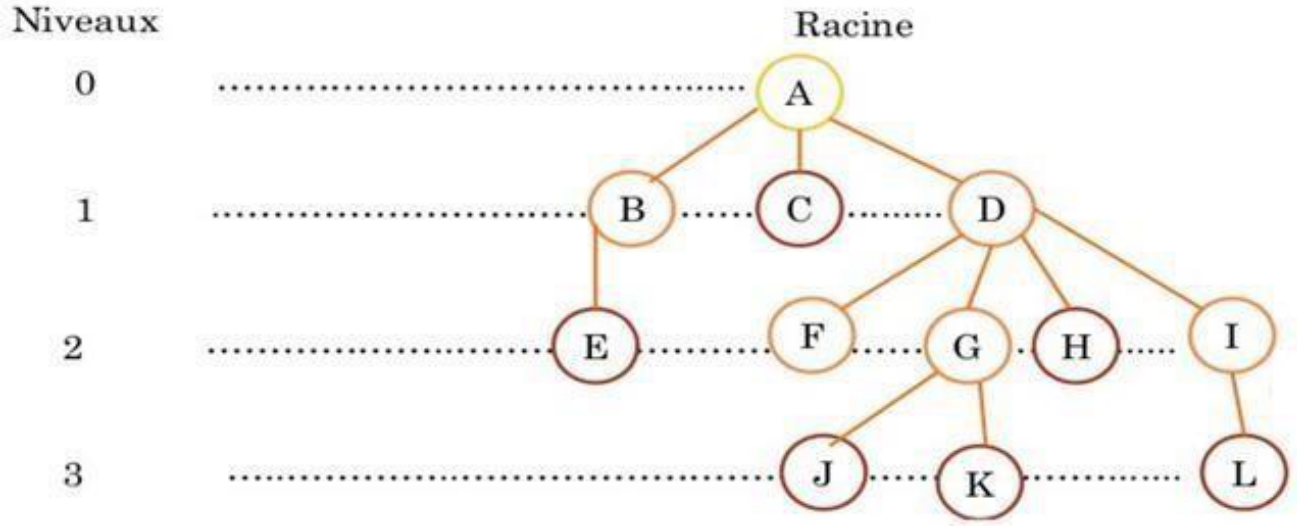
- Le niveau d'un nœud: est la distance qui le sépare de la racine.



Niveau de la racine = 0

Le niveau de chaque nœud est égal au niveau de son père plus 1

Le niveau du nœud contenant G est égal à 2.



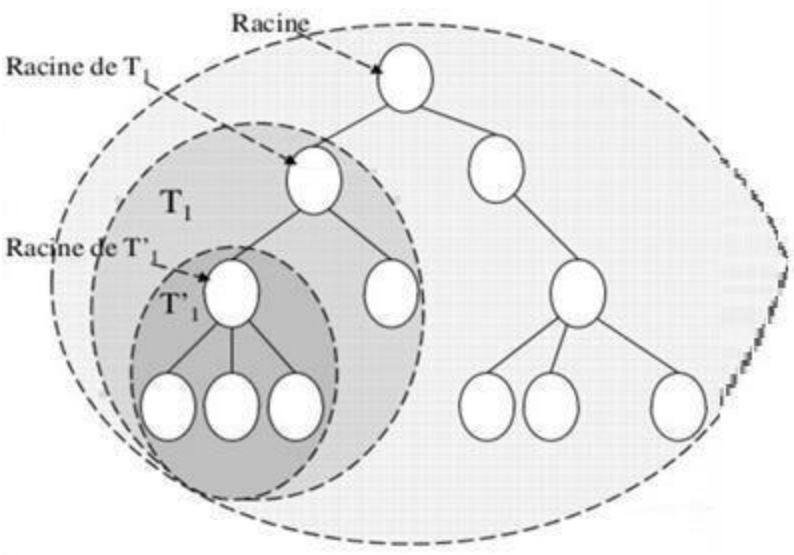
Définition récursive:

Cas particulier: NIL est un arbre vide contenant 0 nœud.

Cas général: si n est un nœud et si T1, T2, …, Tm sont des arbres. Alors on peut construire un nouvel arbre en connectant T1, T2, …, Tm sont des fils à n.

Chaque Ti est défini de la même manière récursivement.

T1, T2, …, tm sont alors des sous- arbres de n.

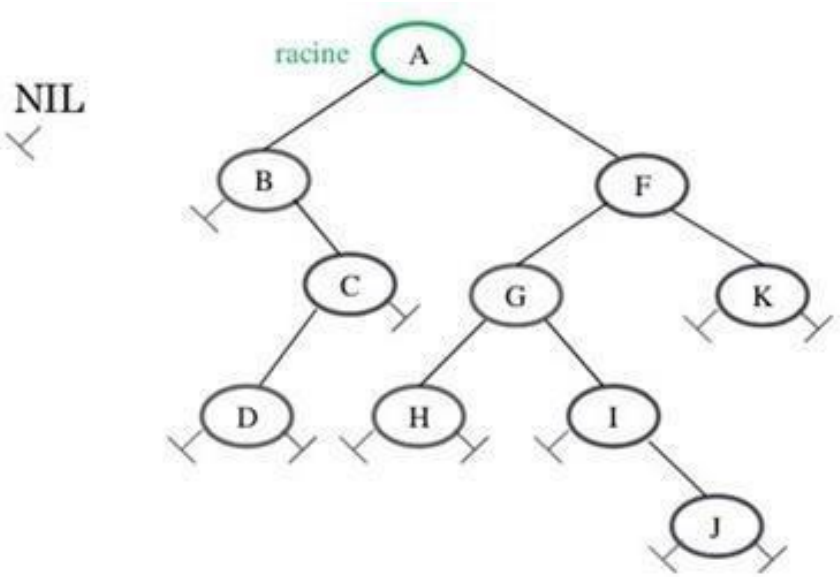


- Arbre m-aire: est un arbre d'ordre m et le degré maximum d'un nœud est égal à m.
- B-arbre: un arbre B d'ordre n est un arbre où:
  - La racine a au moins deux fils
  - Chaque nœud autre que la racine a entre n/2 et n fils
  - Tous les nœuds feuilles sont au même niveau
- Arbre binaire: est un arbre où le degré maximum d'un nœud est égal à 2.
- Arbre binaire de recherche: c'est un arbre binaire où la clé de chaque nœud est supérieure à celle de ses descendants gauches et inférieure à celle des ses descendants droite.

TERMINOLOGIE "ARBRE BINAIRE"

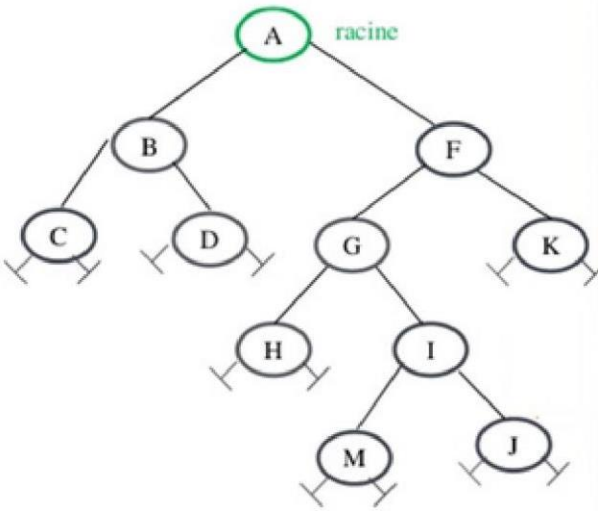
- Un arbre binaire est un arbre où chaque nœud est connecté à deux sous-arbres (un sous-arbre gauche et un sous arbre droit)
- C'est un arbre de degré 2, c'est-à-dire que chaque nœud a au plus deux fils.

➢ Le premier fils d'un nœud n est appelé Fils gauche (FG) et le deuxième fils est appelé Fils droit (FD)

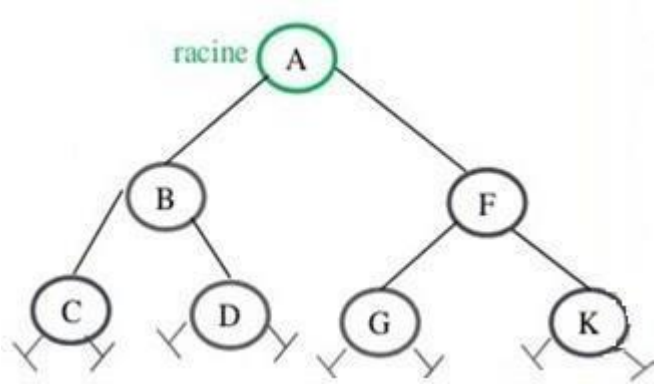


Définition

Un arbre binaire est dit strictement binaire si chaque nœud interne a exactement 2 fils différents de NIL.



Un arbre binaire complet est un arbre strictement binaire où toutes les feuilles sont au même niveau.



- L'arbre est implémenté souvent de manière dynamique comme un ensemble de nœuds chaînés entre eux.

La structure d'un nœud de l'arbre est la suivante:

Types noeud =enregistrement

val : élément //valeur à stocker

FG: pointeur sur nœud // FilsGauche

FD : pointeur sur noeud // FilsDroite

Fin enregistrement

Arbre= pointeur sur noeud

Variables

A: Arbre

- **Création d'un arbre vide**

Cette primitive retourne un arbre vide. Il suffit de déclarer un pointeur sur un arbre binaire et de l'initialiser à Nil.

Fonction Créer_arbre_vide () : Arbre

Var A : Arbre

Début

A←Nil

Retourner (A) Fin

- **Création d'une feuille**

Cette primitive permet de créer un nœud externe contenant une valeur et retourne sa position en mémoire.

Fonction Créer_Feuille (e : élément) : Arbre

Var

Feuille : Arbre

Début

Allouer (Feuille)

Feuille->Val ← e

Feuille-> FG ← Nil

Feuille-> FD ← Nil

Retourner (Feuille)

Fin

Création d'un arbre

Cette primitive permet de créer un arbre à partir d'un sous-arbre droit et d'un sous-arbre gauche existants et d'un élément qui représente la valeur de la racine.

Fonction Créer_arbre (e : élément, SAG: arbre, SAD: Arbre) : Arbre

Var

père : Arbre Début

Allouer (père) père->val ← e père-> FG ← SAG

père-> FD ← SAD

Retourner (père)

Fin

Fonctions de base

- **Tester la vacuité d'un arbre**

Fonction est_arbre_vide (A: arbre) : booleen

Début

Retourner (A =Nil)

Fin

- *Tester si un nœud est une feuille*

Fonction est_Feuille (A: arbre) : booleen

Début

Si (NON est_arbre_vide (A)) alors

Retourner (est_arbre vide (A-> SAG) et est_arbre vide (A-> SAD))

Sinon écrire (« l'arbre est vide »)

Finsi

Fin

Fonctions de base

- **Accès à la racine**

Cette fonction récupère la valeur contenu dans le nœud racine d'un arbre. Pour pouvoir accéder aux valeur des autres nœuds il faut d'abord les transformer en racine. Par exemple , soit x un nœud de l'arbre, pour accéder à la valeur de x, il faut se déplacer dans l'arbre jusqu'à atteindre le sous- arbre qui a pour racine x.

Fonction racine (A: arbre) : élément

Début

Si (NON est_arbre_vide (A)) alors

Retourner (A-> val)

Sinon écrire (« l'arbre est vide »)

Finsi

Fin

Fonctions de base

- **Positionnement sur le fils gauche**

Cette primitive permet d'obtenir la position en mémoire du fils gauche d'un arbre.

Fonction Fils_Gauche (A: arbre) : arbre

Début

Si (NON est_arbre_vide (A)) alors

Retourner (A-> FG)

Sinon écrire (« l'arbre est vide »)

Finsi Fin

- **Positionnement sur le fils droite**

Cette primitive permet d'obtenir la position en mémoire du fils droite d'un arbre.
Fonction Fils_droit (A: arbre) : arbre

Début

Si (NON est_arbre_vide (A)) alors

Retourner (A-> FD)

Sinon écrire (« l'arbre est vide »)

Finsi Fin

Fonctions de base

- **Suppression d'un arbre**

Cette primitive permet de détruire la totalité de l'arbre ou du sous-arbre dont la position en mémoire est fournies en paramètre: cette position est celle de la racine.

Procédure DetruireArbre (A: arbre)

Début

Si (NON est_arbre_vide (A)) alors

DetruireArbre (FilsGauche (A))

DetruireArbre (FilsDroite(A))

Liberer (A)

Finsi Fin

Il est possible de fournir comme paramètre de cette procédure le nœud racine d'un arbre, le nœud racine d'un sous-arbre ou le nœud représentant une feuille.

Applications

- Taille d’un arbre

Le calcul de la taille d’un arbre revient à calculer le nombre de nœuds de cet arbre.

Fonction taille (A: arbre) : entier

Début

si (A = Nil) alors retourner (0)

sinon retourner (1+ taille (A-> FG) + taille (A->FD)) Finsi

Fin

- **Nombre de feuilles d’un arbre**

Fonction nombre_feuille (A: arbre) : entier

Début

si (A = Nil) alors retourner (0)

sinon si (A-> FG = Nil) et (A->FD = Nil) alors retourner 1 sinon

retourner (nombre_feuille (A-> FG) + nombre_feuille (A-> FD)) Finsi Finsi Fin

Applications

- **Recherche d'un élément dans un arbre**

Fonction Recherche (A: arbre, x: entier) : booleen

Début

si (A = Nil) alors retourner (faux)

sinon si (A-> val = x) alors

retourner (vrai) sinon retourner (Recherche (A-> FG, x) ou Recherche (A-> FD, x))
Finsi

Finsi

Fin

Parcours

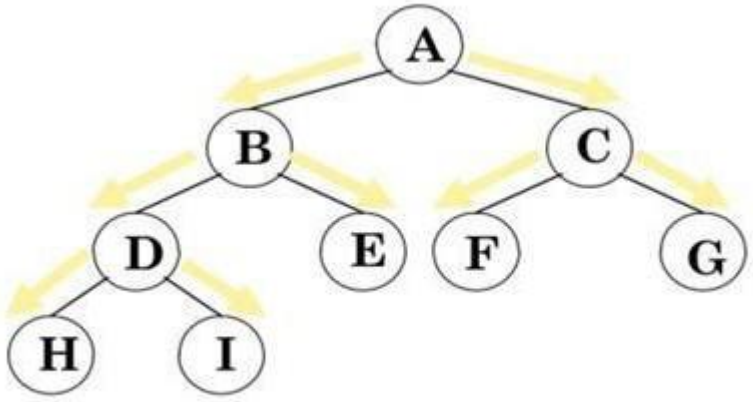
- Le parcours d'un arbre consiste à passer par tous ses nœuds.
- Les parcours permettent d'effectuer tout un ensemble de traitement sur les arbres.
- On distingue deux types de parcours:
  - Des parcours en largeur explorent l'arbre niveau par niveau.
  - Des parcours en profondeur explorent l'arbre branche par branche où on descend le plus profondément possible dans l'arbre puis une fois qu'une feuille a été atteinte, on remonte pour explorer les autres branches en commençant par la branche ''la plus basse'' parmi celles non encore parcourues.

Le parcours en profondeur peut se faire en:

- ✓ le Préordre (Préfixe): où on affiche la racine avant ses fils (R, FG, FD)
- ✓ L'Inordre (Infixe): où on affiche FG puis racine puis FD (FG, R, FD).
- ✓ Le Postordre (Postfixe): où on affiche les fils avant la racine (FG, FD, R).

Parcours en préordre (préfixe)

➢ Le parcours préordre de l'arbre R (s'il n'est pas vide) consiste à visiter le nœud racine (R) ensuite parcourir récursivement en préordre les sous arbres T1 (sous arbre gauche) puis T2 (sous arbre droit) ce qui donne : [R, T1, T2] ou (RGD).



Résultat de parcours: ABDHIEC FG

Il s'agit de parcourir la racine d'abord, ensuite explorer le sous-arbre gauche, et finalement explorer le sous-arbre droit.

La procédure (récursive) qui affiche les valeurs en parcours préfixe d'un arbre A est:

Procédure Préfixe(A: arbre)

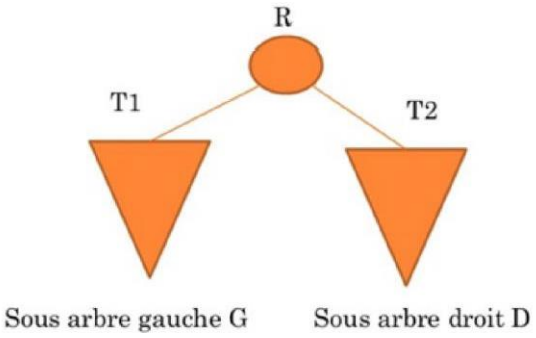
Début

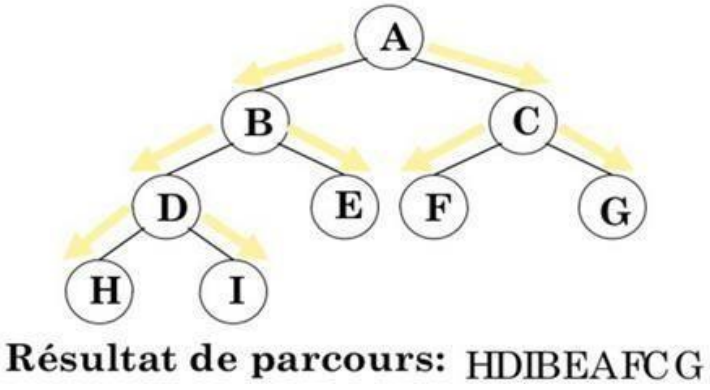
Si (NON est_arbre_vide (A)) alors
Ecrire (Racine (A))
Préfixe (FilsGauche (A))
Préfixe (FilsDroite(A))
Finsi

Fin

Parcours en inordre (infixe)

- Le parcours inordre de l'arbre R (s'il n'est pas vide) consiste à parcourir récursivement en inordre les sous arbres T1 (sous arbre gauche) puis visiter le nœud racine (R) ensuite parcourir récursivement en inordre T2 (sous arbre droit) ce qui donne : [T1, R, T2] ou (GRD)





Il s'agit d'explorer le sous-arbre gauche, visiter la racine et finalement explorer le sous-arbre droit.

La procédure (récursive) qui affiche les valeurs en parcours infixe d'un arbre A est:

Procédure Infixe(A: arbre)

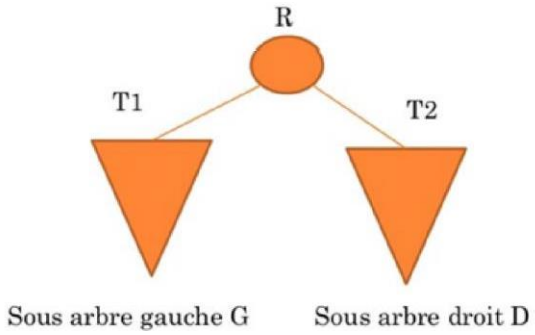
Début

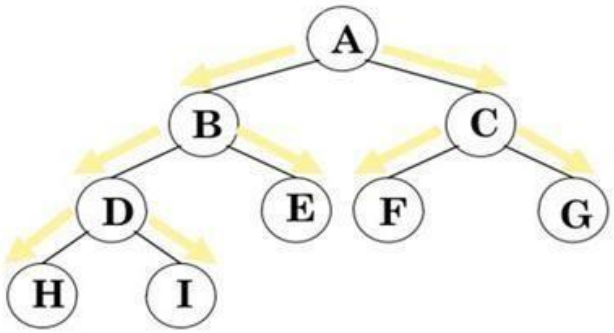
Si (NON est_arbre_vide (A)) alors Infixe (FilsGauche (A))
Ecrire (Racine (A))
Infixe (FilsDroite(A))
Finsi

Fin

Parcours en postordre (postfixe)

➢ Le parcours postordre de l'arbre R (s'il n'est pas vide) consiste à parcourir récursivement en inordre les sous arbres T1 (sous arbre gauche) ensuite parcourir récursivement en inordre T2 (sous arbre droit) puis visiter le nœud racine (R) ce qui donne : [T1, T2, R] ou (GDR)





Résultat de parcours: HIDEBFGCA

Il s'agit d'explorer le sous-arbre gauche, explorer le sous-arbre droit et finalement visiter la racine.

La procédure (récursive) qui affiche les valeurs en parcours postfixe d'un arbre A est:

Procédure Postfixe(A: arbre)

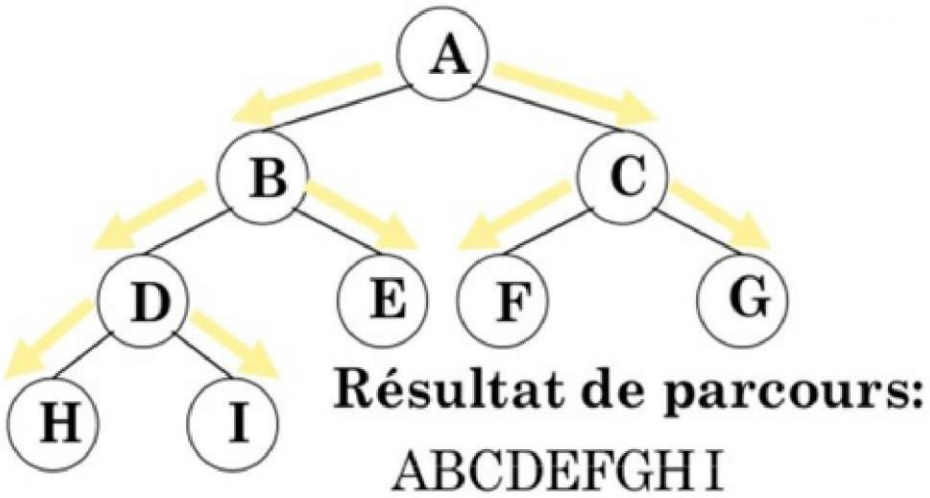
Début

Si (NON est_arbre_vide (A)) alors
Postfixe (FilsGauche (A))
Postfixe (FilsDroite(A))
Ecrire (Racine (A))
Finsi

Fin

Parcours en largeur

- Dans le parcours par niveau, tous les nœuds d'un même niveau sont traités avant de descendre au niveau suivant .
- Il explore les nœuds de l'arbre niveau après niveau, l'exploration se fait dans l'ordre suivant: le noeud racine, les nœuds du niveau 1, les noeuds du niveau 2, …etc.



- Pour ce type de parcours on ne peut pas appliquer la récursivité, car l'arbre n'obéit plus à une définition récursive, mais il est considéré comme étant formés de niveaux, chaque niveau contenant un certain nombre de nœuds.

- Il faut parcourir les nœuds en ordre du haut vers le bas et de la gauche vers la droite. Pour cela il faut utiliser une autre structure de données, que nous appellerons F. Il faut sauvegarder les nœuds visités, en faisant en sorte que les nœuds d'un même niveau soient successifs dans la structure F. C'est-à-dire que le frère (ou le cousin) d'un nœud doit précéder ses enfants dans F.

- Nous utilisons une liste pour fournir les éléments de l'arbre à la sortie de la fonction, mais il est aussi possible de les traiter directement sans les stocker.

- Dans la liste nous allons stocker la valeur contenue dans chaque nœud visité.

- Dans la file nous allons stocker les nœuds de l'arbre.

```
Fonction parcours Largeur (A: arbre) : liste var  L: liste; F: file; B: arbre

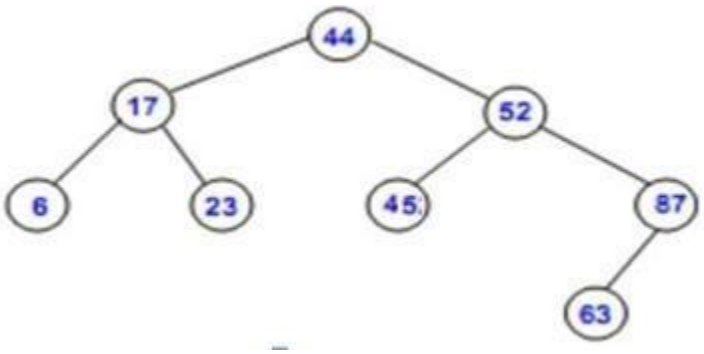
Début

B ← A
F ← créer_file vide ()
L ← créer_Liste vide ()
Enfiler (F, A)
Tant que (NON Filevide (F) ) faire
B ← Lirefile (F) Défiler (F) si (B != Nil) alors
L ← ajouter_fin (L, Racine (B))
Si (B->FG != Nil) alors
Enfiler (F, FilsGauche (B))
Finsi
Si (B->FD != Nil) alors
Enfiler (F, FilsDroit (B))
Finsi
Finsi
 Fintq
Retourner (L)

 Fin
```

TERMINOLOGIE "ARBRE BINAIRE DE RECHERCHE"

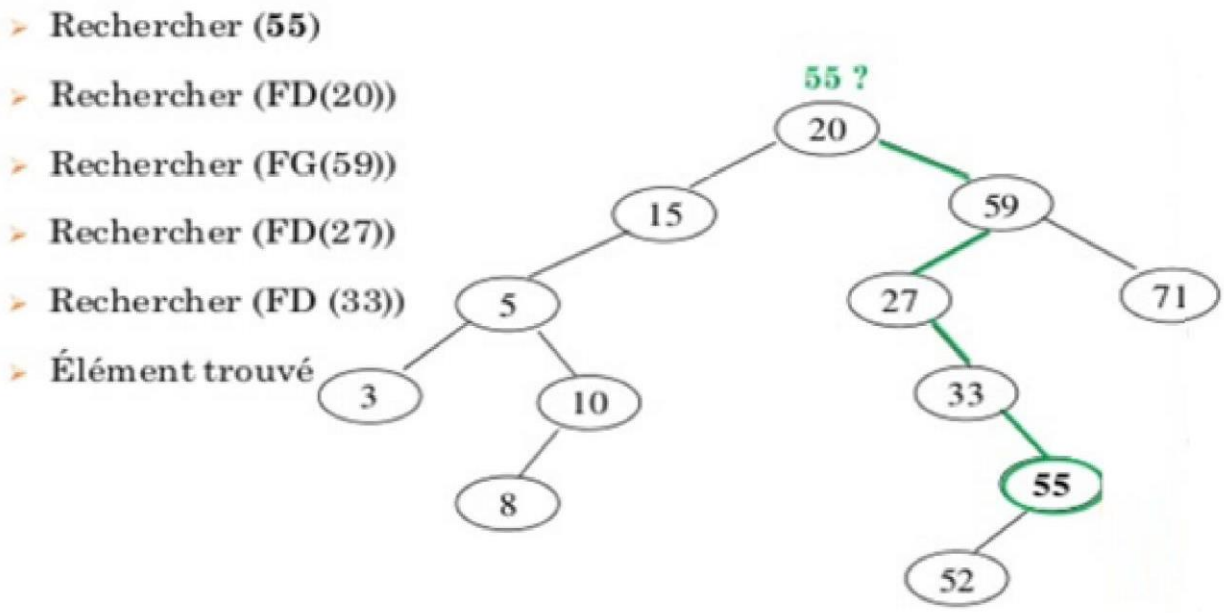
- Un arbre binaire de recherche (ABR) est un arbre binaire ordonné telque pour tout nœud n:
  - ✓ Toutes les valeurs du sous arbre gauche de n sont inférieures ou égales à la valeur de n
  - ✓ Toutes les valeurs du sous arbre droit de n sont supérieures ou égales à la valeur de n



Remarque: Généralement, les valeurs dans un ABR sont uniques; on n'admet pas de répétition de valeur pour éviter les confusions . Mais si jamais ceci arrive : par exemple si un arbre contient deux fois la valeur 4, par convention, la deuxième valeur est stockée dans le sous-arbre droit ayant pour racine 4.

A- Recherche d'un élément dans un ABR

La recherche est dichotomique, à chaque étape, un sous arbre est éliminé:



A- Recherche d'un élément dans un ABR

Cette fonction permet de rechercher un élément x dans un ABR et retourner un booléen. Pour les ABR, les éléments sont stockés de façon à respecter une relation d'ordre, cela rend la recherche plus beaucoup plus efficace que pour les arbre binaires quelconques.

Fonction Recherche_rec (A: arbre, x: entier) : booleen

Début

si (A = Nil) alors retourner (faux) sinon si (A-> val = x) alors

retourner (vrai)

sinon

si (A-> val < x) retourner (Recherche_rec (A-> FD, x))

sinon retourner(Recherche_rec (A-> FG, x))

Finsi

Finsi

Finsi

Fin

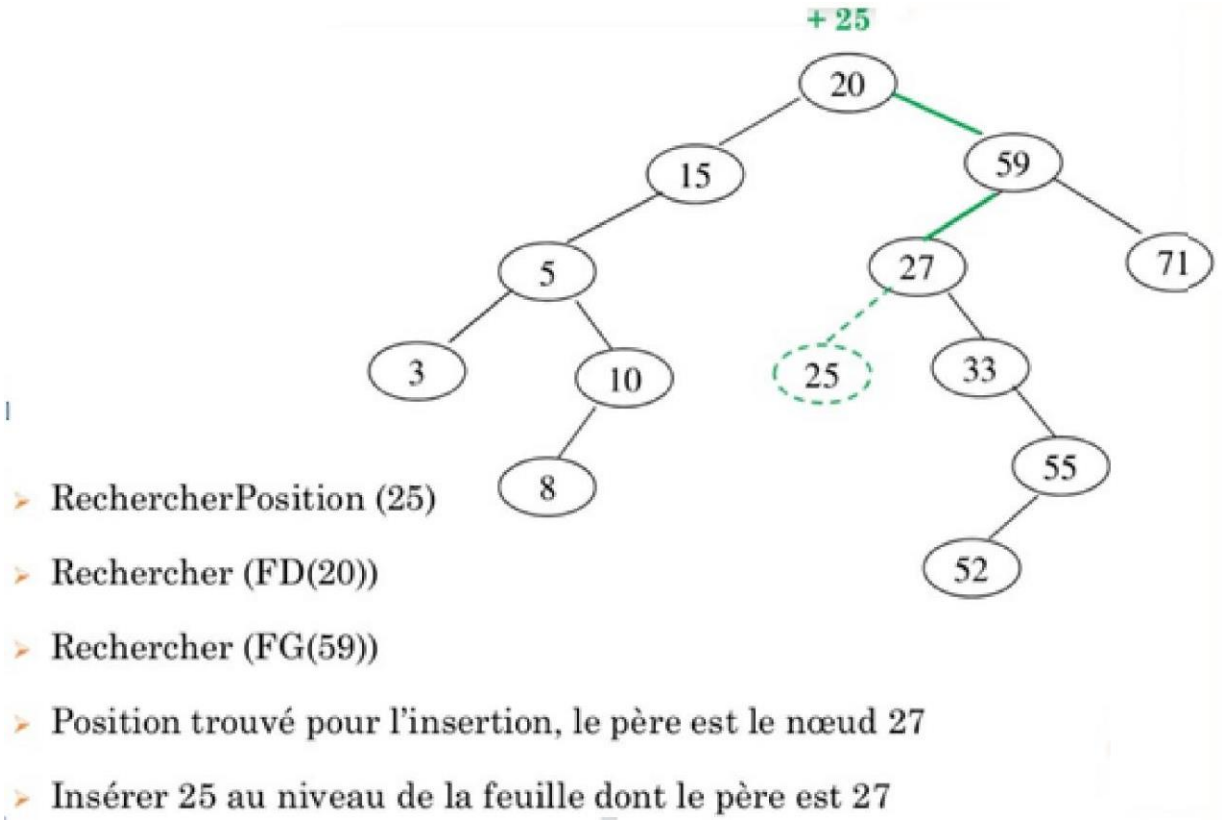
A- Recherche d'un élément dans un ABR (version itérative)

Dans un ABR, on sait de façon précise s'il faut continuer la recherche à gauche ou à droite pour chaque nœud exploré, alors il est possible d'écrire une version itérativede recherche dans un ABR.

```
Fonction Recherche_iter (A: arbre, x:
entier) : booleen
Début
Tant que (A != Nil) et (x != Racine (A)) faire si (A-> val < x) alors
A ← A-> FD
sinon
A ← A-> FG
Finsi
Fintq
si (A = Nil) alors retourner (faux)
sinon retourner (vrai)
Finsi
 Fin
```

B- Insertion d'un élément dans un ABR

Pour insérer un nouvel élément dans un ABR il faut d'abord repérer sa place dans l'arbre, il faut donc le comparer aux éléments déjà existants dans l'ABR. Enfin l'insérer comme fils du dernier nœud visité.



C- Insertion d'un élément dans un ABR

Pour insérer un nouvel élément e dans un ABR il faut d'abord repérer sa place dans l'arbre, il faut donc le comparer aux éléments déjà existants dans l'ABR. Enfin l'insérer comme fils du dernier nœud visité.

```
Procédure Inserer_rec (var A: arbre, e: entier)
Var
P: arbre
Début
si  (A = Nil) alors
allouer (A)
A-> val ← e
A-> FG  ← Nil
A-> FD ← Nil
sinon
si (A-> val  > e) alors
Inserer_rec ( A-> FG, e)
sinon
```

```
Inserer_rec ( A-> FD, e)
Finsi
Finsi
Fin
```

B- Insertion d'un élément dans un ABR (version itérative)

```
Procédure Inserer_iter (var A: arbre, e: entier)
Var
p,père : arbre
Début

Tant que  (P != Nil)  faire
père ← P
si (P-> val > e) alors
P ← P-> FG
sinon
P ← P-> FD
Finsi
Fintq
allouer (P)
P-> val ← e
P-> FG  ← Nil
P-> FD ← Nil

si (père = Nil) alors
A ← P
sinon
si (Père -> val > e ) alors
père  -> FG ← P
Sinon
père  -> FD ← P
FinSI
Finsi

Fin
```

C- Suppression d'un élément d'un ABR

- La suppression dans un ABR est assez compliqué, c'est pour cela que nous allons détailler tous les cas possibles.

- Pour supprimer un nœud dans un ABR, plusieurs cas de figure peuvent se présenter. Il est toutefois nécessaire d'obtenir un ABR à l'issue de la suppression.

- D'abord il faut chercher l'élément à supprimer, une fois trouvé on se trouve dans l'une des situations suivantes, soit « i » le nœud à supprimer:

- 1er cas : i est une feuille : on la supprime et on la remplace par Nil.

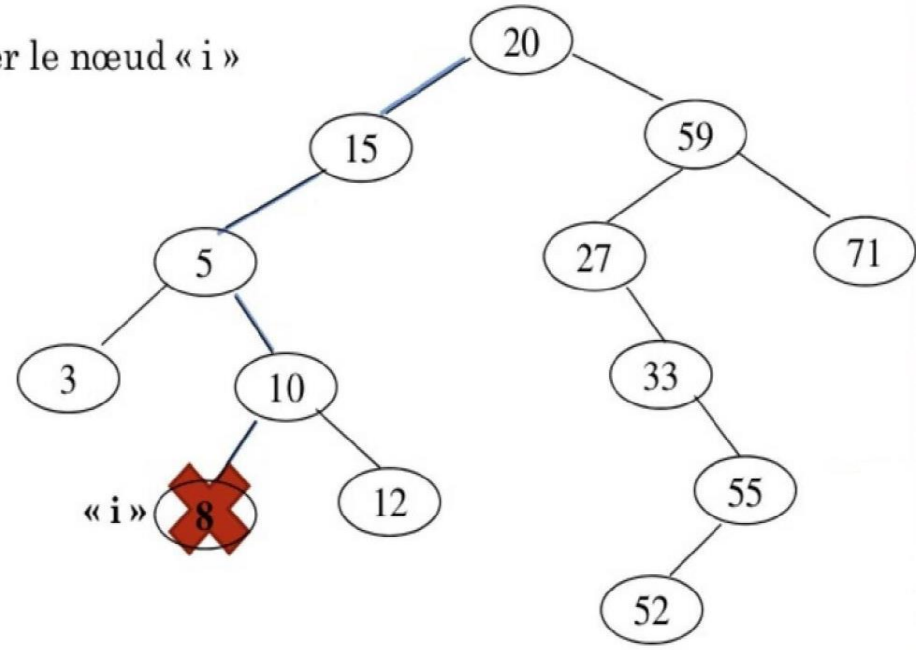
- 2eme cas : i est un nœud qui a un seul fils : on supprime i et on le remplace par ce fils.

- 3eme cas : i est un nœud qui a deux fils : on supprime i et on le remplace par l'élément minimum se trouvant dans son sous arbre droit (le nœud le plus à gauche du sous arbre droit)ou par l'élément maximum se trouvant dans son sous arbre gauche (le nœud le plus à droite du sous arbre gauche).

Cas 1: Suppression d'une feuille

- Il suffit de l'enlever de l'arbre vu qu'elle n'a pas de fils.
- Exemple: supprimer le nœud i qui contient la valeur 8

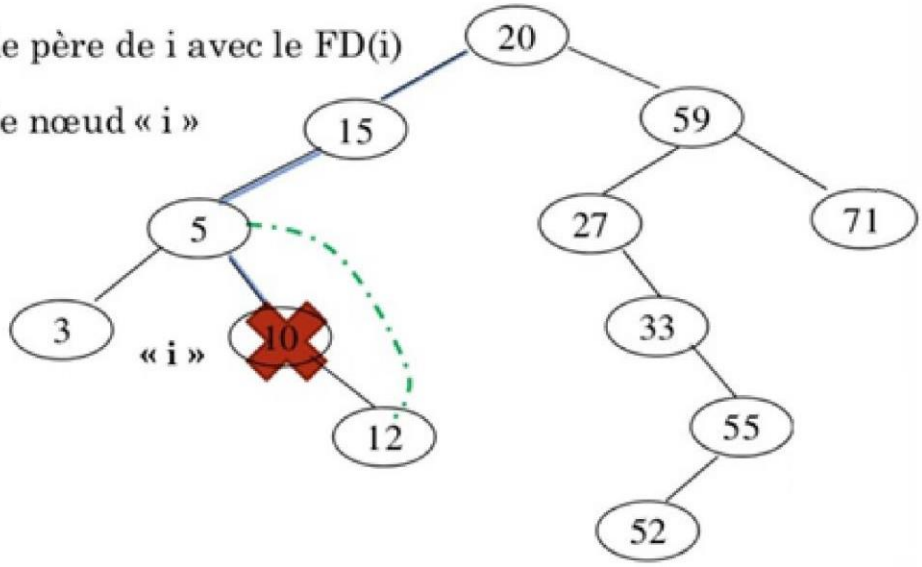
1. Rechercher(8)
2. Libérer le nœud « i »



Cas 2: Suppression d'un nœud avec un fils

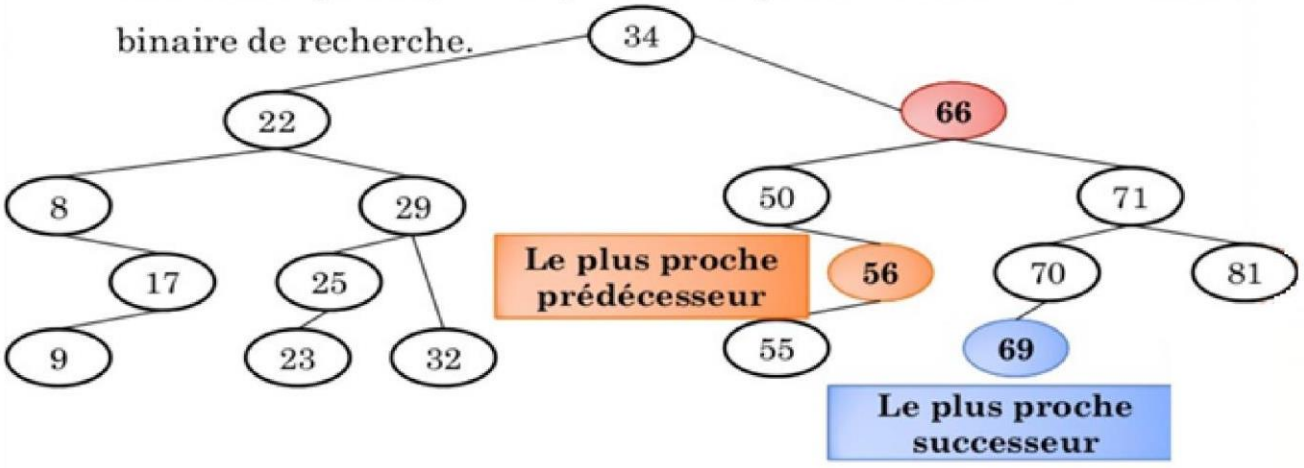
- Il faut l'enlever de l'arbre en le remplaçant par son fils.
- Exemple: supprimer le nœud i qui contient la valeur 10

1. Rechercher(**10**)
2. Chainer le père de i avec le FD(i)
3. Libérer le nœud « i »



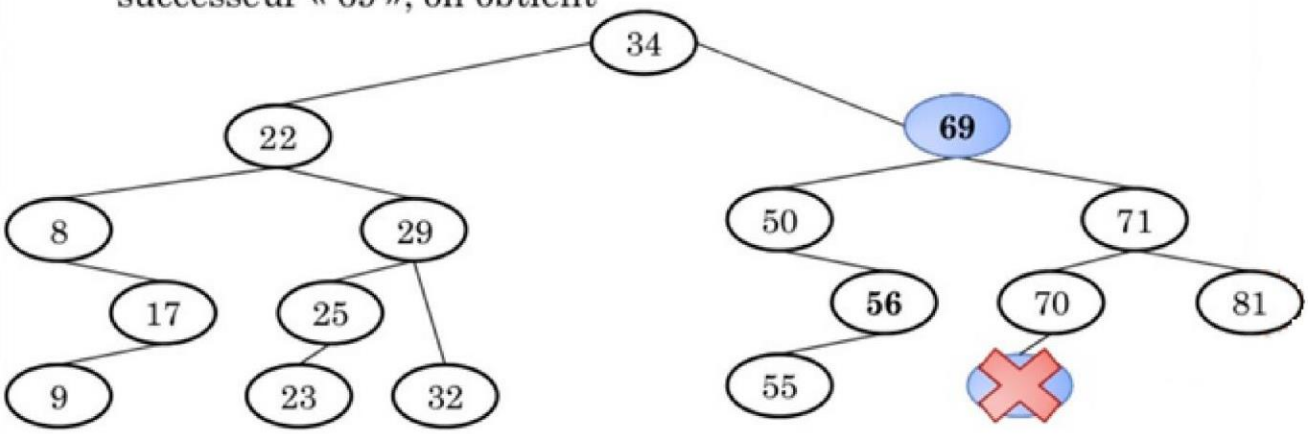
Cas 3: Suppression d'un nœud avec deux fils

- **Etape 1:** On échange le nœud à supprimer avec **son successeur le plus proche** (le nœud le plus à gauche du sous-arbre droit) ou **son plus proche prédécesseur** (le nœud le plus à droite du sous-arbre gauche). Cela permet de garder une structure d'arbre binaire de recherche.



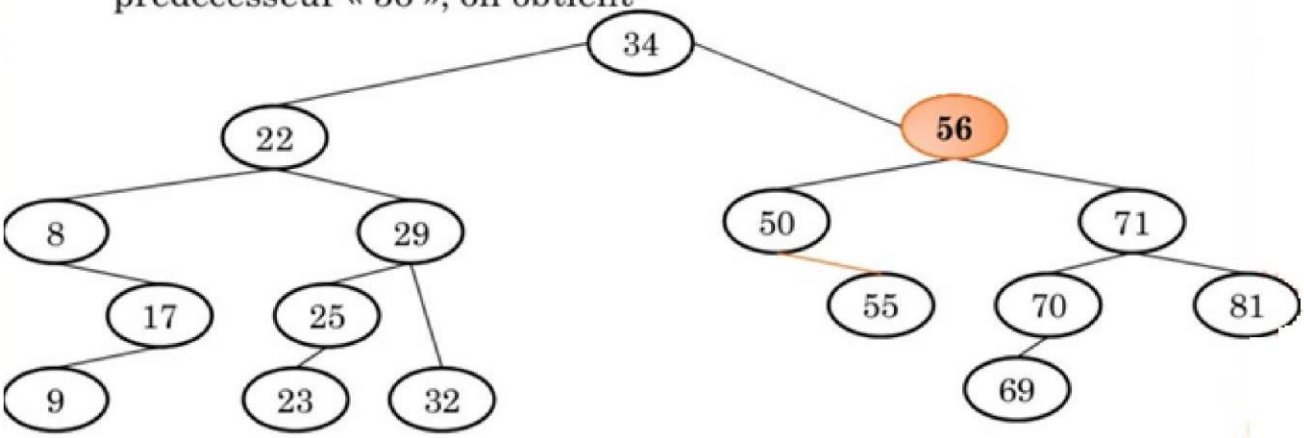
Cas 3: Suppression d'un nœud avec deux fils

- **Etape 2:** on applique à nouveau la procédure de suppression qui est maintenant une feuille ou un nœud avec un seul fils.
- Ainsi, si on choisit d'échanger le nœud « 66 » avec son plus proche successeur « 69 », on obtient



Cas 3: Suppression d'un nœud avec deux fils

- Puis on applique à nouveau la procédure de suppression qui est maintenant une feuille ou un nœud avec un seul fils.
- Ainsi, si on choisit d' échanger le nœud « 66 » avec son plus proche prédécesseur « 56 », on obtient



Pour le 3ème cas nous avons besoin d'abord de déterminer le plus proche prédécesseur (maximum du SAG du nœud i) et le plus proche successeur (minimum du SAD du nœud i).

- Fonction maximum (A: arbre): arbre

Début
si (A-> FD != Nil) alors
retourner (maximum (A-> FD))
sinon
retourner (A)
Finsi
Fin

- Fonction minimum (A: arbre): arbre

Début
si (A-> FG != Nil) alors
retourner (minimum (A-> FG))
sinon
retourner (A) Finsi

Fin

NB: pour la fonction maximum A, sera remplacé par FG(i) et pour minimum, A sera remplacé par FD(i).

Chapitre 06 : Algorithmique des Listes.

Listes chaînées

1. Structures de données linéaires

Parmi les structures de données linéaires il y a :

- les tableaux,
- les listes chaînées,
- les piles,
- les files.

Les structures de données linéaires induisent une notion de séquence entre les éléments les composant (1er, 2ème, 3ème, suivant, dernier…).

1.1. Les tableaux

Vous connaissez déjà la structure linéaire de type **tableau** pour lequel les éléments de même type le composant sont placés de façon contigüe en mémoire.
Pour créer un tableau, à 1 ou 2 dimensions, il faut connaître sa taille qui ne pourra être modifiée au cours du programme, et lui associer un indice pour parcourir ses éléments.
Pour les tableaux la séquence correspond aux numéros des cases du tableau. On accède à un élément du tableau directement grâce à son indice.

Soit le tableau à 1 dimension suivant nommé Tablo :

12	14	10	24	

Pour atteindre la troisième case du tableau il suffit d'écrire Tablo[3] qui contient 10, si les valeurs de l'indice commencent à 1.

La structure de type tableau pose des problèmes pour insérer ou supprimer un élément car ces actions nécessitent des décalages du contenu des cases du tableau qui prennent du temps dans l'exécution d'un programme.

Ce type de stockage de valeurs peut donc être coûteux en temps d'exécution. Il existe

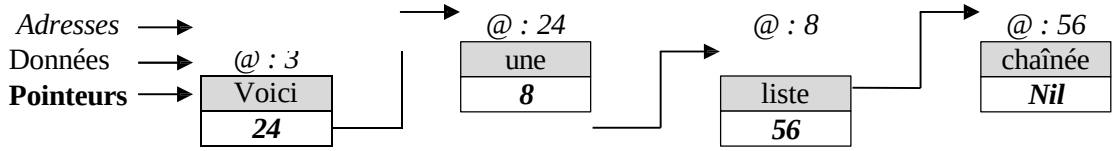
une autre structure, appelée liste chaînée, pour stocker des valeurs, cette structure permet plus aisément d'insérer et de supprimer des valeurs dans une liste linéaire d'éléments.

1.2. Les listes chaînées

Une **liste chaînée** est une structure linéaire qui n'a pas de dimension fixée à sa création. Ses éléments de même type sont éparpillés dans la mémoire et reliés entre eux par des pointeurs. Sa dimension peut être modifiée selon la place disponible en mémoire. La liste est accessible uniquement par sa tête de liste c’est-à-dire son premier élément.

Pour les listes chaînées la séquence est mise en oeuvre par le pointeur porté par chaque élément qui indique l'emplacement de l'élément suivant. Le dernier élément de la liste ne pointe sur rien (*Nil*). On accède à un élément de la liste en parcourant les éléments grâce à leurs pointeurs.

Soit la liste chaînée suivante (@ indique que le nombre qui le suit représente une adresse) :



Pour accéder au troisième élément de la liste il faut toujours débuter la lecture de la liste par son premier élément dans le pointeur duquel est indiqué la position du deuxième élément. Dans le pointeur du deuxième élément de la liste on trouve la position du troisième élément…
Pour ajouter, supprimer ou déplacer un élément il suffit d'allouer une place en mémoire et de mettre à jour les pointeurs des éléments.

Il existe différents types de listes chaînées :

- **Liste chaînée simple** constituée d'éléments reliés entre eux par des pointeurs.
- **Liste chaînée ordonnée** où l'élément suivant est plus grand que le précédent. L'insertion et la suppression d'élément se font de façon à ce que la liste reste triée.
- **Liste doublement chaînée** où chaque élément dispose non plus d'un mais de deux pointeurs pointant respectivement sur l'élément précédent et l'élément suivant. Ceci permet de lire la liste dans les deux sens, du premier vers le dernier élément ou inversement.
- **Liste circulaire** où le dernier élément pointe sur le premier élément de la liste. S'il s'agit d'une liste doublement chaînée alors de premier élément pointe également sur le dernier.

Ces différents types peuvent être mixés selon les besoins.

On utilise une liste chaînée plutôt qu'un tableau lorsque l'on doit traiter des objets représentés par des suites sur lesquelles on doit effectuer de nombreuses suppressions et de nombreux ajouts. Les manipulations sont alors plus rapides qu'avec des tableaux.

Résumé

Structure	Dimension	Position d'une information	Accès à une information
Tableau	Fixe	Par son indice	Directement par l'indice
Liste chaînée	Evolue selon les actions	Par son adresse	Séquentiellement par le pointeur de chaque élément

1.3. Les piles et les files

Les **files** et les **piles** sont des listes chaînées particulières qui permettent l'ajout et la suppression d'éléments uniquement à une des deux extrémités de la liste.

Structures	Ajout	Suppression	Type de Liste
PILE	Tête	Tête	LIFO (Last In First Out)
FILE	Queue	Tête	FIFO (First In First Out)

La pile est une structure de liste similaire à une pile d'assiettes où l'on pose et l'on prend au sommet de la pile.
La file est une structure de liste similaire à une file d'attente à une caisse, le premier client entré dans la file est le premier sorti de celle-ci (aucun resquillage n'est admis).

2. Listes chaînées

2.1. Définitions

Un **élément** d'une liste est l'ensemble (ou structure) formé :

- d'une donnée ou information,
- d'un pointeur nommé *Suivant* indiquant la position de l'élément le suivant dans la liste.

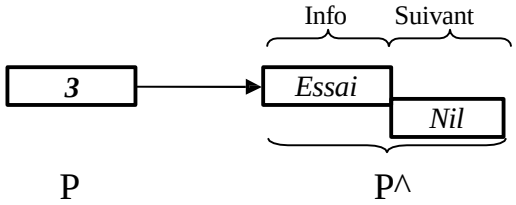
A chaque élément est associée une adresse mémoire.

Les listes chaînées font appel à la notion de variable dynamique. Une variable dynamique:

- est déclarée au début de l'exécution d'un programme,
- elle y est créée, c'est-à-dire qu'on lui alloue un espace à occuper à une adresse de la mémoire,
- elle peut y être détruite, c'est-à-dire que l'espace mémoire qu'elle occupait est libéré,
- l'accès à la valeur se fait à l'aide d'un pointeur.

Un **pointeur** est une variable dont la valeur est une adresse mémoire (voir chapitre 9). Un pointeur, noté P, pointe sur une variable dynamique notée P^.
Le **type de base** est le type de la variable pointée.
Le **type du pointeur** est l'ensemble des adresses des variables pointées du type de base. Il est représenté par le symbole ^ suivi de l'identificateur du type de base.

Exemple:



La variable pointeur P pointe sur l'espace mémoire P^ d'adresse 3. Cette cellule mémoire contient la valeur "*Essai*" dans le champ *Info* et la valeur spéciale *Nil* dans le champ *Suivant*. Ce champ servira à indiquer quel est l'élément suivant lorsque la cellule fera partie d'une liste. La valeur Nil indique qu'il n'y a pas d'élément suivant. **P^** est l'objet dont l'adresse est rangée dans P.

Les listes chaînées entraînent l'utilisation de procédures d'allocation et de libération dynamiques de la mémoire. Ces procédures sont les suivantes:

- **Allouer(P)** : réserve un espace mémoire P^ et donne pour valeur à P l'adresse de cet espace mémoire. On alloue un espace mémoire pour un élément sur lequel pointe P.
- **Désallouer(P)** : libère l'espace mémoire qui était occupé par l'élément à supprimer P^ sur lequel pointe P.

Pour définir les variables utilisées dans l'exemple ci-dessus, il faut :

- définir le type des éléments de liste : Type Cellule= Structure
  Info : Chaîne
  Suivant : Liste
  fin Structure
- définir le type du pointeur : Type Liste = ^Cellule

- déclarer une variable pointeur : Var P : Liste
- allouer une cellule mémoire qui réserve un espace en mémoire et donne à P la valeur de l'adresse de l'espace mémoire P^ : Allouer(P)
- affecter des valeur à l'espace mémoire P^: P^.Info ← "Essai" ; P^.Suivant← Nil

Quand P = Nil alors P ne pointe sur rien.

2.2. Listes chaînées simples

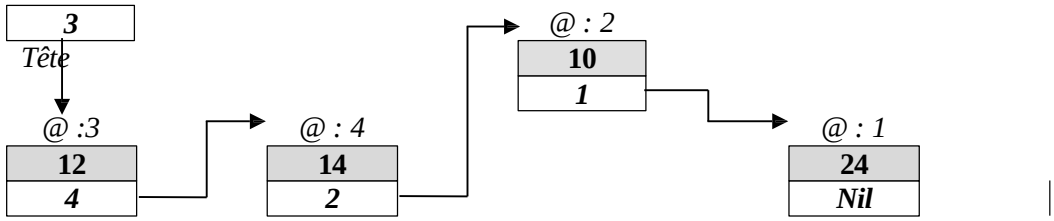
Une liste chaînée simple est composée :

- d'un ensemble d'éléments tel que chacun :
  - est rangé en mémoire à une certaine adresse,
  - contient une donnée (*Info*),
  - contient un pointeur, souvent nommé *Suivant*, qui contient l'adresse de l'élément suivant dans la liste,
- d'une variable, appelée *Tête*, contenant l'adresse du premier élément de la liste chaînée.

Le pointeur du dernier élément contient la valeur *Nil*. Dans le cas d'une liste vide le pointeur de la tête contient la valeur *Nil*. Une liste est définie par l'adresse de son premier élément.

Avant d'écrire des algorithmes manipulant une liste chaînée, il est utile de montrer un schéma représentant graphiquement l'organisation des éléments de la liste chaînée.

Exemple:
Reprenons l'exemple du tableau paragraphe 1.1. page 1. La liste chaînée correspondante pourrait être :



Le 1er élément de la liste vaut 12 à l'adresse 3 (début de la liste chaînée)

Le 2e élément de la liste vaut 14 à l'adresse 4 (car le pointeur de la cellule d'adresse 3 est égal à 4) Le 3e élément de la liste vaut 10 à l'adresse 2 (car le pointeur de la cellule d'adresse 4 est égal à 2) Le 4e élément de la liste vaut 24 à l'adresse 1 (car le pointeur de la cellule d'adresse 2 est égal à 1)

Si P a pour valeur 3	**Si P a pour valeur 2**
P^.Info a pour valeur 12	P^.Info a pour valeur 10
P^.Suivant a pour valeur 4	P^.Suivant a pour valeur 1

2.3. Traitements de base d'utilisation d'une liste chaînée simple

Il faut commencer par définir un type de variable pour chaque élément de la chaîne. En

langage algorithmique ceci se fait comme suit :

```
Type Liste = ^Element
Type Element =
Structure
                    Info :  variant
                    Suivant :
                            Liste
                Fin structure
```

Variables Tete, P : Liste
Le type de *Info* dépend des valeurs contenues dans la liste : entier, chaîne de caractères, variant pour un type quelconque…

Les traitements des listes sont les suivants :

- Créer une liste.
- Ajouter un élément.
- Supprimer un élément.
- Modifier un élément.
- Parcourir une liste.
- Rechercher une valeur dans une liste.

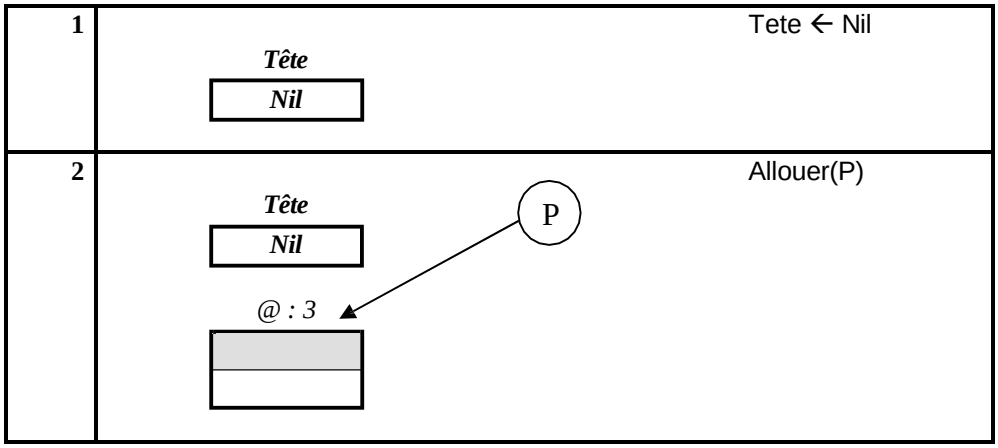
2.3.1 Créer une liste chaînée composée de 2 éléments de type chaîne de caractères

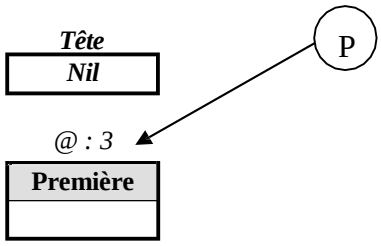
Déclarations des types pour la liste :

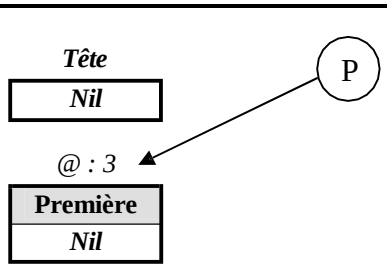
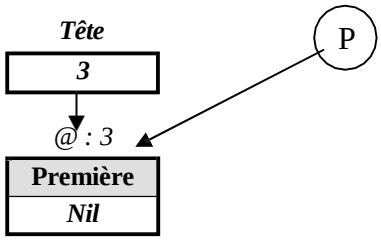
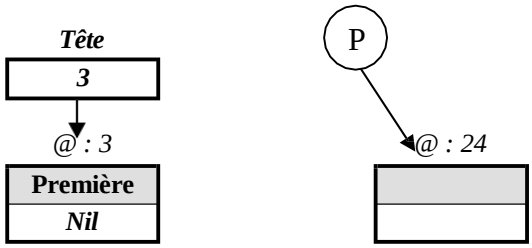
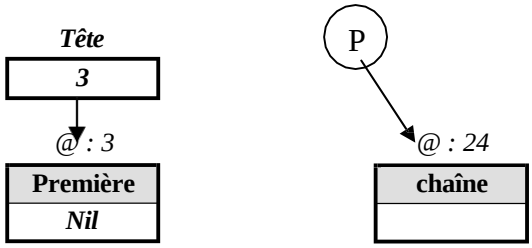
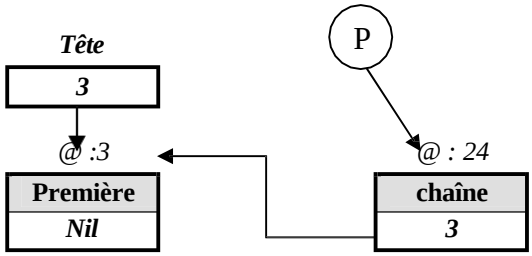
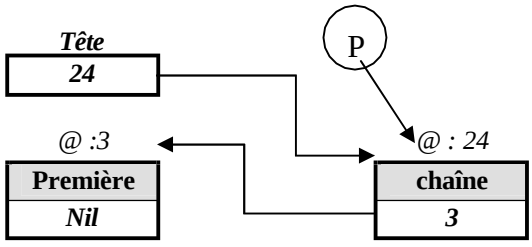
```
Type Liste = ^Element
Type Element =
Structure
        Info :  chaîne de
        caractères Suivant :
                Liste
        Fin structure
```

Algorithme CréationListe2Elements

```
    Tete, P : Liste
    NombreElt : entier
DEBUT
1   Tete ← Nil                          /* pour l'instant la liste est vide*/
2   Allouer(P)                          /* réserve un espace mémoire pour le premier élément */
3   Lire(P^.Info)                       /* stocke dans l'Info de l'élément pointé par P la valeur saisie */
4   P^.Suivant ← Nil                    /* il n'y a pas d'élément suivant */
5   Tete ← P                            /* le pointeur Tete pointe maintenant sur P */
    /* Il faut maintenant ajouter le 2e élément, ce qui revient à insérer un élément en tête de liste */
6   Allouer(P)                          /* réserve un espace mémoire pour le second élément */
7   Lire(P^.Info)                       /* stocke dans l'Info de l'élément pointé par P la valeur saisie */
8   P^.Suivant ← Tete                   /* élément inséré en tête de liste */
9   Tete ← P
FIN
```



3	Lire(P^.Info) 

4	P^.Suivant ← Nil 
5	Tete ← P 
6	Allouer(P) 
7	Lire(P^.Info) 
8	P^.Suivant ← Tete 
9	Tete ← P 

2.3.2 Créer une liste chaînée composée de plusieurs éléments de type chaîne de caractères

Déclarations des types pour la liste :

```
Type Liste = ^Element
Type Element =
Structure
        Info : chaîne de caractères
        Suivant : Liste
fin Structure
```

Pour créer une liste chaînée contenant un nombre d'éléments à préciser par l'utilisateur il suffit d'introduire deux variables de type Entier *NombreElt* et *Compteur*

- de faire saisir la valeur de *NombreElt* par l'utilisateur dès le début du programme,
- d'écrire une boucle Pour *Compteur allant de 1 à NombreElt* comprenant les instructions 6, 7, 8 et 9.

```
Algorithme CréationListeNombreConnu
    Tete, P : Liste
    NombreElt : entier
    Compteur : entier
DEBUT
    Lire(NombreElt)
    Tete ← Nil
    POUR Compteur DE 1 A NombreElt FAIRE
        Allouer(P)              /* réserve un espace mémoire pour l'élément à ajouter */
        Lire(P^.Info)           /* stocke dans l'Info de l'élément pointé par P la valeur saisie */
        P^.Suivant ← Tete       /* élément inséré en tête de liste */
        Tete ← P                /* le pointeur Tete pointe maintenant sur P */
    FIN POUR
FIN
```

Pour créer une liste chaînée contenant un nombre indéterminé d'éléments il faut :

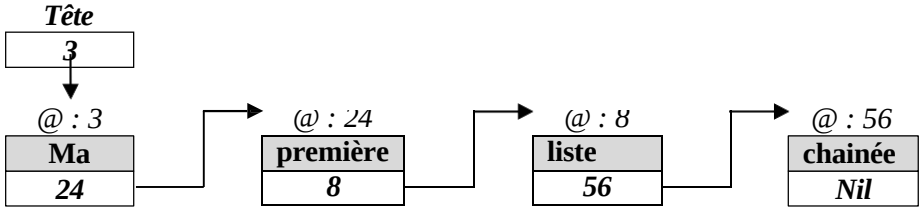
- déclarer une variable de lecture de même type que celui de l'information portée par la liste,
- déterminer et indiquer à l'utilisateur la valeur qu'il doit saisir pour annoncer qu'il n'y a plus d'autre élément à ajouter dans la chaîne (ici "XXX"),
- écrire une boucle Tant Que permettant d'exécuter les instructions 6, 7, 8 et 9 tant que la valeur saisie par l'utilisateur est différente de la valeur indiquant la fin de l'ajout d'élément dans la chaîne.

```
Algorithme CréationListeNombreInconnu
    Tete, P : Liste
    Valeur : chaîne de caractères
DEBUT
    Tete ← Nil
    Lire (Valeur)
    TANT QUE que Valeur ≠ "XXX" FAIRE
        Allouer(P)              /* réserve un espace mémoire pour l'élément à ajouter */
        P^.Info ← Valeur        /* stocke dans l'Info de l'élément pointé par P la valeur saisie */
        P^.Suivant ← Tete       /* élément inséré en tête de liste */
```

```
        Tete ← P                    /* le pointeur Tete pointe maintenant sur P */
        Lire (Valeur)
    FIN TANT QUE
FIN
```

2.3.3. Afficher les éléments d'une liste chaînée

Une liste chaînée simple ne peut être parcourue que du premier vers le dernier élément de la liste.



L'algorithme est donné sous forme d'une procédure qui reçoit la tête de liste en paramètre.

```
Procedure AfficherListe  (Entrée        P : Liste)
/* Afficher les éléments d'une liste chaînée passée en paramètre */
  DEBUT
1 |   P ← Tete                  /* P pointe sur le premier élément de la liste*/
  |                              /* On parcourt la liste tant que l'adresse de l'élément suivant n'est pas Nil */
  |   TANT QUE P <> NIL FAIRE                /* si la liste est vide Tete est à Nil */
2 |       Ecrire(P^.Info)                    /* afficher la valeur contenue à l'adresse pointée par P */
3 |       P ← P^.Suivant                     /* On passe à l'élément suivant */
  |   FIN TANT QUE
  FIN
```

1	P a pour valeur 3
2	"Ma" s'affiche
3	P prend pour valeur 24
2	"première" s'affiche
3	P prend pour valeur 8
2	"liste" s'affiche
3	P prend pour valeur 56
2	"chaînée" s'affiche
3	P prend pour valeur Nil

On s'arrête puisque P a pour valeur Nil et que c'est la condition d'arrêt de la boucle Tant Que.

2.3.4. Rechercher une valeur donnée dans une liste chaînée ordonnée

Dans cet exemple nous reprenons le cas de la liste chaînée contenant des éléments de type chaine de caractères, mais ce pourrait être tout autre type, selon celui déterminé à la création de la liste (rappelons que tous les éléments d'une liste chaînée doivent avoir le même type). La liste va être parcourue à partir de son premier élément (celui pointé par le pointeur de tête). Il a deux cas d'arrêt :

- avoir trouvé la valeur de l'élément,
- avoir atteint la fin de la liste.

L'algorithme est donné sous forme d'une procédure qui reçoit la tête de liste en paramètre. La valeur à chercher est lue dans la procédure.

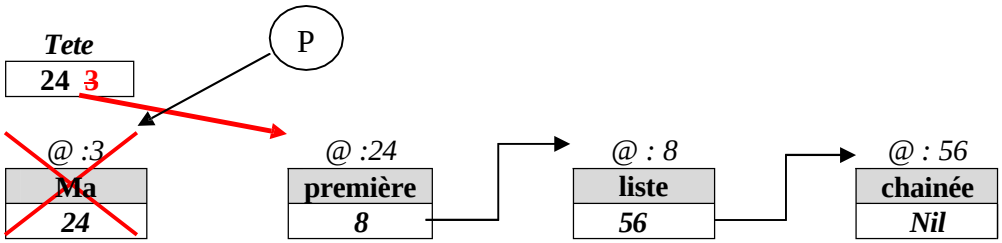
```
Procedure RechercherValeurListe      (Entrée Tete : Liste, Val : variant)
/* Rechercher si une valeur donnée en paramètre est présente dans la liste passée en paramètre */
Variables locales
        P               : Liste      /* pointeur de parcours de la liste */
        Trouve          : booléen    /* indicateur de succès de la recherche */
DEBUT
        SI Tete <> Nil ALORS         /* la liste n'est pas vide on peut donc y chercher une valeur */
                P ← Tete
                Trouve ← Faux
                TANTQUE P <> Nil ET Non Trouve
                        SI P^.Info = Val ALORS        /* L'élément recherché est l'élément courant */
                                Trouve ← Vrai
                        SINON                         /* L'élément courant n'est pas l'élément recherché */
                                P ← P^.Suivant        /* on passe à l'élément suivant dans la liste */
                        FINSI
                FIN TANT QUE
                SI Trouve ALORS
                        Ecrire (" La valeur ", Val, " est dans la liste")
                SINON
                        Ecrire (" La valeur ", Val, " n'est pas dans la liste")
                FINSI
        SINON
                Ecrire("La liste est vide")
        FINSI
FIN
```

2.3.5. Supprimer le premier élément d'une liste chaînée

Il y a deux actions, dans cet ordre, à réaliser :

- faire pointer la tête de liste sur le deuxième élément de la liste,
- libérer l'espace mémoire occupé par l'élément supprimé.

Il est nécessaire de déclarer un pointeur local qui va pointer sur l'élément à supprimer, et permettre de libérer l'espace qu'il occupait.



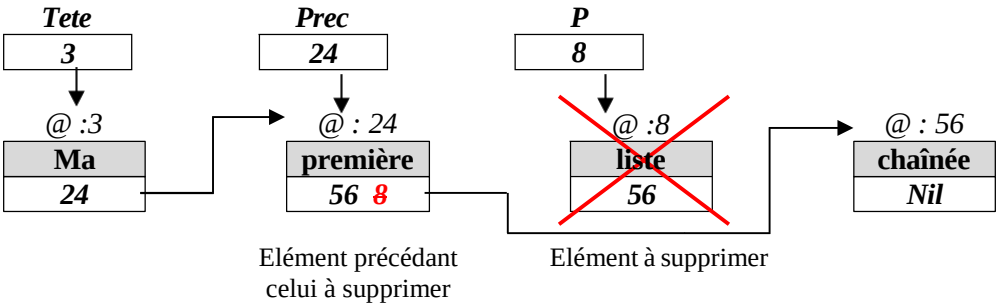

```
Procedure SupprimerPremierElement        (Entrée/Sortie Tete : Liste)
/* Supprime le premier élément de la liste dont le pointeur de tête est passé en paramètre */
Variables locales
        P : Liste                                  /* pointeur sur l'élément à supprimer */
DEBUT
        SI Tete <> Nil ALORS         /* la liste n'est pas vide on peut donc supprimer le premier élément */
                P ← Tete             /* P pointe sur le 1er élément de la liste */
                Tete ← P^.Suivant    /* la tête de liste doit pointer sur le deuxième 'élément */
                Desallouer(P)        /* libération de l'espace mémoire qu'occupait le premier élément */
        SINON
                Ecrire("La liste est vide")
        FINSI
FIN
```

2.3.6. Supprimer d'une liste chaînée un élément portant une valeur donnée

Il faut:

- traiter à part la suppression du premier élément car il faut modifier le pointeur de tête,
- trouver l'adresse *P* de l'élément à supprimer,
- sauvegarder l'adresse *Prec* de l'élément précédant l'élément pointé par *P* pour connaître l'adresse de l'élément précédant l'élément à supprimer, puis faire pointer l'élément précédent sur l'élément suivant l'élément à supprimer,
- Libérer l'espace mémoire occupé par l'élément supprimé.

L'exemple considère que l'on souhaite supprimer l'élément contenant la valeur "liste" de la liste ci- dessus.




```
Procedure SupprimerElement  (Entrée/Sortie Tete : Liste, Val : variant)
/* Supprime l'élément dont la valeur est passée en paramètre */
Variables locales
        P       : Liste                 /* pointeur sur l'élément à supprimer */
        Prec    : Liste                 /* pointeur sur l'élément précédant l'élément à supprimer */
        Trouvé : Liste                  /* indique si l'élément à supprimer a été trouvé */
DEBUT
        SI Tete <> Nil ALORS    /* la liste n'est pas vide on peut donc y chercher une valeur à supprimer */
                SI Tete^.info = Val ALORS               /* l'élément à supprimer est le premier */
                        P ← Tete
                        Tete ← Tete^Suivant
                        Desallouer(P)
                SINON                                   /* l'élément à supprimer n'est pas le premier */
                        Trouve ← Faux
                        Prec ← Tete                     /* pointeur précédent */
                        P ← Tete^.Suivant               /* pointeur courant */
                        TANTQUE P <> Nil ET Non Trouve
                                SI P^.Info = Val ALORS  /* L'élément recherché est l'élément courant */
                                        Trouve ← Vrai
                                SINON                   /* L'élément courant n'est pas l'élément cherché */
                                        Prec ← P        /* on garde la position du précédent */
                                        P^ ← P^.Suivant /* on passe à l'élément suivant dans la liste */
                                FINSI
                        FIN TANT QUE
                        SI Trouve ALORS
                                Prec^.Suivant ← P^.Suivant              /* on "saute" l'élément à supprimer "/
                                Desallouer(P)
                        SINON
                                Ecrire ("La valeur ", Val, " n'est pas dans la liste")
                        FINSI
                FINSI
        SINON

                Ecrire("La liste est vide")
        FINSI
FIN
```

2.4. Listes doublement chaînées

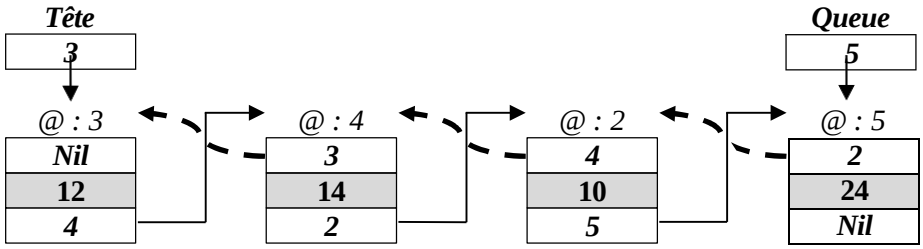
Il existe aussi des liste chaînées, dites bidirectionnelles, qui peuvent être parcourues dans les deux sens, du 1er élément au dernier et inversement.

Une liste chaînée bidirectionnelle est composée :

- d'un ensemble de données,
- de l'ensemble des adresses des éléments de la liste,
- d'un ensemble de pointeurs *Suivant* associés chacun à un élément et qui contient l'adresse de l'élément suivant dans la liste,
- d'un ensemble de pointeurs *Precedent* associés chacun à un élément et qui contient l'adresse de l'élément précédent dans la liste,
- du pointeur sur le premier élément *Tete*, et du pointeur sur le dernier élément, *Queue*,

```
Type ListeDC =
^Element Type
Element =  Structure
                    Precedent :
                    ListeDC Info :
                              variant
                    Suivant :
                              ListeD
                    C
                Fin Structure
```

Le pointeur *Precedent* du premier élément ainsi que le pointeur *Suivant* du dernier élément contiennent la valeur *Nil*.



A l'initialisation d'une liste doublement chaînée les pointeurs *Tete* et *Queue* contiennent la valeur
Nil.

Afficher les éléments d'une liste doublement chaînée

Il est possible de parcourir la liste doublement chaînée du premier élément vers le dernier. Le pointeur de parcours, *P*, est initialisé avec l'adresse contenue dans *Tete*. Il prend les valeurs successives des pointeurs *Suivant* de chaque élément de la liste. Le parcours s'arrête lorsque le pointeur de parcours a la valeur *Nil*. Cet algorithme est analogue à celui du parcours d'une liste simplement chaînée.

```
Procedure AfficherListeAvant   (Entrée      Tete : ListeDC)
Variables locales
        P : ListeDC
  DEBUT
  |     P ← Tete /
  |     TANT QUE P <> NIL FAIRE
  |           Ecrire(P^.Info)
  |           P ← P^.Suivant
  |     FIN TANT QUE
  FI
```

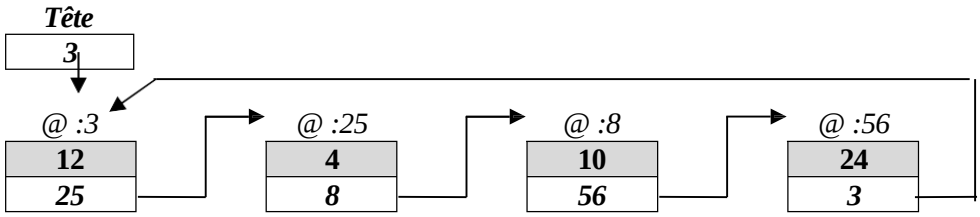
Il est possible de parcourir la liste doublement chaînée du dernier élément vers le premier. Le pointeur de parcours, *P*, est initialisé avec l'adresse contenue dans *Queue*. Il prend les valeurs successives des pointeurs *Precedent* de chaque élément de la liste. Le parcours s'arrête lorsque le pointeur de parcours a la valeur *Nil*.

```
Procedure AfficherListeArriere (Entrée      Queue : ListeDC)
Variables locales
        P : ListeDC
DEBUT
        P ← Queue
        TANT QUE P <> NIL FAIRE
                Ecrire(P^.Info)
                P ← P^.Precedent
        FIN TANT QUE
FIN
```

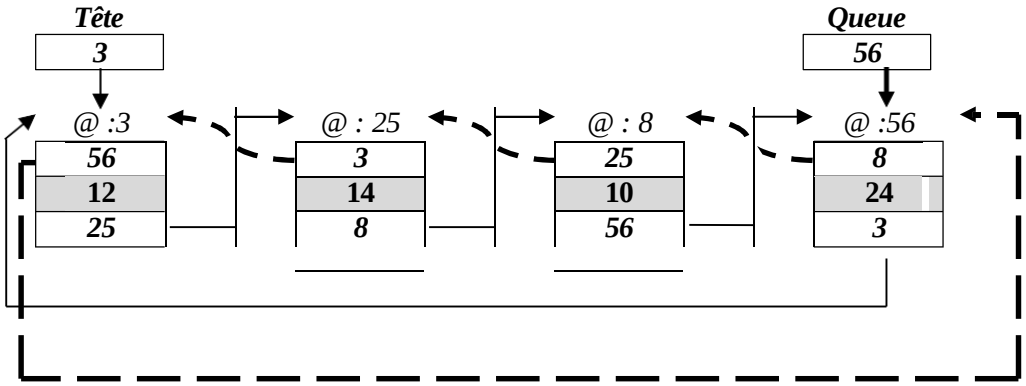
2.5. Listes chaînées circulaires

Une liste chaînée peut être circulaire, c'est à dire que le pointeur du dernier élément contient l'adresse du premier.

Ci-dessous l'exemple d'une liste simplement chaînée circulaire : le dernier élément pointe sur le premier.



Puis l'exemple d'une liste doublement chaînée circulaire. : Le dernier élément pointe sur le premier, et le premier élément pointe sur le dernier.



WEBOGRAPHIE

http://www.siteduzero.com/tutoriel-3-36245-les-listes-chainees.html http://liris.cnrs.fr/pierre-

antoine.champin/enseignement/algo/listes_chainees/
http://deptinfo.cnam.fr/Enseignement/CycleA/SD/cours/structuress%E9quentielleschain%E9es.pdf http://pauillac.inria.fr/~maranget/X/421/poly/listes.html#toc2
http://wwwens.uqac.ca/~rebaine/8INF805/courslistespilesetfiles.pdf